

RENANCE

FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN

CSC 312 — Past Questions & Materials
by Renance

Course: CSC 312
Institution: Federal University of Petroleum Resources, Effurun
Papers in this pack: 3
Levels: 300

This study pack was compiled and restructured by Renance for CSC 312 students at Federal University of Petroleum Resources, Effurun. It brings together 3 past paper(s)/material(s) into one clean, readable document so you can revise without chasing scattered PDFs. Practise these same questions interactively on Renance with instant answers, AI explanations, and full CBT mock exams — over 235,000+ questions across JAMB, Post-UTME, WAEC, NECO and university courses.

Study online: renance.app
Get the app: Google Play — [dev.renance.renance](https://play.google.com/store/apps/details?id=dev.renance.renance)

Generated by Renance AI

Paper 1 of 3 — CSC 312 PAST QUESTION

Level 300 • First Semester • PastQuestion

PETROLEUM RESOURCES, EFFURUN

2021/2022 First SEMESTER EXAMINATION

ent algorithms for dynamic Partitioning and explain how

May arise from them {5mks}

(Given memory Partitions of 100 KB, 500 KB, 200 KB, 300 kB and 600 KB (in order), how would each of the Placement algorithms listed in

(b) above place processes of 212KB, 417 KB, 112 KB

4nd 426 KB (in that Order) ? Which algorithm makes the

Most efficient use of memory? (6mks)

Question Two

fa) Outline three Responsibilities of the Operating system with respect to memory management {6mks}

main memory must accommodate both operat

Briefly explain how this is

Question Three

(a) Consider the following segment table:

Base length

0 219 600

1 2300 14

2 90 100

'J 1327 580

4 1952 96

What are the physical addresses for the following logical addresses? (10mks)

a. 0,430

b.1,10

c. 2,500

d. 3,400

@. 4,112

ing system and the various user spaces,

achieved by the operating system (3mmks)

(b) Explain the hardware Sipe" that enables the operating system to distinguish between Pages in memory and pages on disk in virtual memory (mks)

Page 1of2

Question Four

FOFANG an tof t isted ring the questior n-pre base all

a a format ha t th r re d ust

processe 2mks)

Question Five

(i) Race condition in mutual exclusive kernel

(6). Race condition in computer system | a bank system that

maintains an account has functions deposit(amount) and draw(amount)

The function that is to be deposited or withdrawn from the

bank. Assume that there is a bank account. Concurrently

it is possible that might be done to prevent the race

Question Six

(a). the critical section problem has four conditions that must hold to have a good

Briefly discuss the tools for solving the critical section

i) State one limitation of Peterson's solution to the critical section problem (2mks)

Page 20f

Paper 2 of 3 — CSC312: OPERATING SYSTEM II [ORHIONKPAIYO B. C]

Level 300 • First Semester • Handout

CSC312: Operating System II

Department of Computer Science

Federal University of Petroleum Resources, Effurun,

Delta State

CSC312:

OPERATING SYSTEM II

[ORHIONKPAIYO B. C]

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 1

CSC312: Operating System II

1. COURSE GOAL

The goal of this course is to extend the students' fundamental knowledge in operating systems. It intends to acquaint students with the mechanisms involved in process and memory management in contemporary Operating Systems.

2. COURSE OBJECTIVES

At the end of this course, students should be able to:

- i. Describe the various ways of organizing memory hardware.
- ii. Explain various techniques of allocating memory to processes.
- iii. Discuss in detail how paging works in contemporary computer systems.
- iv. Discuss both software and hardware solutions of the critical-section problem.
- v. Examine some classical process-synchronization problems.
- vi. Examine the tools that are used to solve process synchronization problems
- vii. Describe various CPU-scheduling algorithms.
- viii. Discuss evaluation criteria for selecting a CPU-scheduling algorithm for a particular system.
- ix. Examine the scheduling algorithms of some operating systems.
- x. Discuss the placement algorithms of some memory management techniques
- xi. Explain the causes and types of fragmentation and their possible solutions
- xii. Differentiate between logical address and physical address
- xiii. Explain the term race condition and state its causes
- xiv. State the critical section problem and illustrate the different methods for solving it
- xv. Enumerate the conditions that must hold to achieve a good solution to critical section problem
- xvi. Different between semaphore and mutex locks and illustrate how each solves the critical section problem
- xvii. State the adverse effect of busy waiting in multiprogramming systems
- xviii. Explain the cause of deadlock and illustrate the concept using two processes

- xix. List the classical problems that are commonly used to illustrate the power of synchronization primitives
- xx. Illustrate classical synchronization problems using appropriate synchronization primitives

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 2

CSC312: Operating System II

3. COURSE DESCRIPTION

Concurrency: States & State diagrams Structures, Dispatching and Context

Switching; interrupts; Concurrent execution; Mutual exclusion problem and some solutions

Deadlock; Models and mechanisms (Semaphores, monitors etc.)

Producer – Consumer Problems & Synchronization. Multiprocessor issues. Scheduling & Despatching. Memory Management: Overlays, Swapping and Partitions, Paging & Segmentations Placement & replacement policies, working sets and Trashing, Caching.

4. COURSE SCHEDULE

WEEK TOPICS SUB TOPICS

1 Course Orientation and the syllabus Review of course syllabus

2– 4 Memory Management i. Memory management requirement

ii. Contiguous memory allocation

iii. Multiple partitioning

iv. Swapping

v. Paging

vi. Segmentation

vii. Virtual Memory

5 –7 CPU Scheduling i. Types of CPU scheduling

ii. Criteria for Selecting a CPU-scheduling algorithm

iii. Scheduling algorithms

a. First-Come, First-Served

Scheduling

b. Shortest-Job-First Scheduling

c. Priority Scheduling

d. Round-Robin Scheduling

e. Multi-level Queue Scheduling

f. Multilevel feedback Queue

Scheduling

8 - 10 Mutual Exclusion & Synchronization i. Principles of concurrency

ii. Mutual exclusion hardware support

iii. Semaphores

iv. Monitors

v. Message passing

vi. Reader/Writer problem

11 - 13 Deadlock & Starvation i. Principles of deadlock

ii. Deadlock prevention

iii. Deadlock Avoidance

iv. Dining Philosophers problem

14 Test & Revision

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 3

CSC312: Operating System II

5. CONTACTS

6. GRADING

Attendance - 3%

Assignments/Laboratory - 15%

Test - 12%

Exam - 70%

7. POLICIES

(i) Attendance - It is expected that every student will be in class for lectures. Attendance records will be kept and used to award marks for each student. Only students with up to or above 75% attendance will be awarded 3mks. Signing for friends is an act of DISHONESTY and it is HIGHLY PROHIBITED.

(ii) Makeup test - There shall be no makeup test. The date for the test shall be announced to the class at least one week before the test.

(iii) Academic Integrity - Any form of academic dishonesty, such as copying during test /exam or making copies of other students' work, is prohibited.

(iv). Submission of Assignments/Lab work - Late submission of assignment or lab work will be graded as Score - x, where x is the number of days of late submission.

9. TEXTBOOKS

1. Operating System Concepts, Abraham Silberchatz, Peter B. Galvin, Greg Gagne, Ninth Edition, 2012, Wiley.

2. Operating Systems: Internals & Design Principles - Williams Stallings

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 4

CSC312: Operating System II

1. MEMORY MANAGEMENT

The main memory is central to the operation of a modern computer system. The CPU loads instructions only from memory, so any programs to run must be stored there. Main memory is a repository of quickly accessible data shared by the CPU

and I/O devices.

The operating system is responsible for the following activities in connection with memory management:

- Keeping track of which parts of memory are currently being used and who is using them
- Deciding which processes (or parts of processes) and data to move into and out of memory
- Allocating and deallocating memory space as needed

Thus, memory management is the task carried out by the Operating System and hardware to accommodate multiple processes in main memory. If only a few processes can be kept in main memory, then much of the time all processes will be waiting for I/O and the CPU will be idle.

o The purpose of memory management is to ensure fair, secure, orderly, and efficient use of memory.

To improve both the utilization of the CPU and the speed of the computer's response to its users,

- o general-purpose computers must keep several programs in memory, creating a need for memory management.
- o thus, memory needs to be allocated efficiently in order to have as many processes in memory as possible
- o Many different memory management schemes are used. These schemes reflect various approaches, and the effectiveness of any given algorithm depends on the situation.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 5

CSC312: Operating System II

- In most schemes, the kernel occupies some fixed portion of main memory and the rest is shared by multiple processes

General-purpose computers run most of their programs from rewritable memory, called main memory (also called Random-Access Memory, or RAM). The programs and data do not reside in main memory permanently. This arrangement usually is not possible for the following two reasons:

1. Main memory is usually too small to store all needed programs and data permanently.
2. Main memory is a volatile storage device that loses its contents when power is turned off or otherwise lost.

Thus, most computer systems provide secondary storage as an extension of main memory, thus providing virtual memory system

- o Virtual memory allows the computer to use part of a permanent storage device (such as a hard disk) as extra memory.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 6

CSC312: Operating System II

1.1 Memory Management Requirements

It is helpful to keep in mind the requirements that memory management intended to satisfy. These requirements include the following:

- i. Relocation
- ii. Protection
- iii. Sharing
- iv. Logical organization
- v. Physical organization

□ Relocation

The memory available has to be shared among various processes present in a multiprogramming system. Thus, it is impossible to determine what other programs would reside in the main memory during the program execution.

If the active processes are swapped in and out of the main memory, it lets the OS have a bigger and better pool of processes that are ready to execute.

Whenever any program happens to swap out to its disk memory, it would not always be possible that swapping it back into the main memory would let it occupy the previous memory location. It is because the location might still be occupied by some other process. The given processes might have to relocate to a different section of the memory.

After the program gets loaded into the main memory, the OS and the processor should be able to translate the logical addresses into the physical addresses

□ The address generated by the CPU is said to be a logical address. An address generated by MMU is called a physical address

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 7

CSC312: Operating System II

□ Protection

Every process has to be protected against all unwanted interferences when all the other processes try to write into a process, whether it is accidental or incidental.

Processes should not be able to reference memory locations in another process without permission. It must be checked at run time.

The memory protection requirement must be satisfied by the processor (hardware) rather than the operating system (software).

The operating can protect the memory with the help of base and limit register.

The requirement of memory protection has to be satisfied by the processor instead of the OS, since the OS can hardly control any given process whenever it occupies the processor.

□ Sharing

Some protection mechanisms allow various processes to access a similar section of main memory. Thus, it allows all the processes to access the very same copy of the program instead of having their own separate copy, which has an advantage.

For instance, numerous processes may utilize the very same system file. It is natural to load a single copy of the given file in the main memory and to let it be shared by these processes.

o It is the memory management's task to allow the shared areas of memory with controlled access without compromising the protection.

Various mechanisms are utilized here to support the relocation-supported sharing abilities.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 8

CSC312: Operating System II

□ Logical Organization

The main memory is organized as a linear form, or it could be an address space that's one-dimensional. It comprises a sequence of words or bytes.

Programs are written in modules. Modules can be written and compiled independently.

o Different degrees of protection are given to modules (read-only, execute-only). Modules are shared among processes.

□ Physical Organization

A computer memory's structure has two levels, and they are referred to as the main memory and the secondary memory. The secondary memory is mainly used for data storage on a long-term basis, but the main memory holds the programs currently in use.

o moving information between these two levels of memory is a major concern of memory management (OS)

□ it is highly inefficient to leave this responsibility to the application programmer

□ Memory must be fairly allocated for high processor utilization for systematic flow of information between main and secondary memory

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 9

CSC312: Operating System II

1.2 Basics of memory Management

In order to understand the concept of memory management, we will discuss some issues that are pertinent to managing memory:

- basic hardware,
- binding symbolic memory addresses to actual physical addresses,
- the distinction between logical and physical addresses, and
- dynamic linking and shared libraries.

1.2.1 Basic Hardware

Memory & Registers:

Main memory and the registers built into the processor itself are the only general-purpose storage that the CPU can access directly.

All forms of memory provide an array of bytes. Each byte has its own address.

Interaction is achieved through a sequence of load or store instructions to specific memory addresses.

- o The load instruction moves a byte or word from main memory to an internal register within the CPU, whereas
- o The store instruction moves the content of a register to main memory

There are machine instructions that take memory addresses as arguments, but none that take disk addresses.

- o Therefore, any instructions in execution, and any data being used by the instructions, must be in one of these direct-access storage devices.

If the data are not in memory, they must be moved there before the CPU can operate on them.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 10

CSC312: Operating System II

Cache:

Cache sits between main memory and CPU registers. Registers that are built into the CPU are generally accessible within one cycle of the CPU clock. Completing a memory access may take many cycles of the CPU clock.

- o In such cases, the processor normally needs to stall, since it does not have the data required to complete the instruction that it is executing.

□ This situation is intolerable because of the frequency of memory accesses.

- o The remedy is to add fast memory (or cache) between the CPU and main memory, typically on the CPU chip for fast access.

Base and Limit Registers: A pair of base and limit registers define the logical address space.

- o The base register holds the smallest legal physical memory address;
- o The limit register specifies the size of the range.

For example, if the base register holds 300040 and the limit register is 120900, then the program can legally access all addresses from 300040 through 420939 (inclusive).

CPU must check every memory access generated in user mode to be sure it is between base and limit for that user, thus protecting the memory space

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 11

CSC312: Operating System II

- o Any attempt by a program executing in user mode to access operating-system memory or other users' memory results in a trap to the operating system, which treats the attempt as a fatal error (see diagram below)

The base and limit registers can be loaded only by the operating system, which uses a special privileged instruction.

- o Since privileged instructions can be executed only in kernel mode, and since only the operating system executes in kernel mode, only the operating system can load the base and limit registers.

- o This scheme prevents a user program from (accidentally or deliberately) modifying the code or data structures of either the operating system or other users.

1.2.2 Address Binding

Usually, a program resides on a disk as a binary executable file. To be executed, the program must be brought into memory and placed within a process.

- o The processes on the disk that are waiting to be brought into memory for execution form the input queue or job queue.

The normal single-tasking procedure is to select one of the processes in the input queue and to load that process into memory.

- o As the process is executed, it accesses instructions and data from memory.

- o Eventually, the process terminates, and its memory space is declared available.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 12

CSC312: Operating System II

Addresses in the source program are generally symbolic (such as the variable count).

- o A compiler typically binds these symbolic addresses to relocatable addresses

- o The linkage editor or loader in turn binds the relocatable addresses to absolute addresses.

Each binding is a mapping from one address space to another.

Classically, the binding of instructions and data to memory addresses can be done at any step along the way:

- Compile time: If memory location known a priori, absolute code can be generated; must recompile code if starting location changes
 - o For example, if you know that a user process will reside starting at location R, then the generated compiler code will start at that location and extend up from there. If, at some later time, the starting location changes, then it will be necessary to recompile this code.
- Load time: Must generate relocatable code if memory location is not known at compile time
- Execution time: Binding delayed until run time if the process can be moved during its execution from one memory segment to another. Need hardware support for address maps (e.g., base and limit registers).

1.2.3 Logical Versus Physical Address Space

An address generated by the CPU is commonly referred to as a logical address also referred to as virtual address, whereas an address seen by the memory unit—that is, the one loaded into the memory-address register of the memory—is commonly referred to as a physical address.

- o Logical and physical addresses are the same in compile-time and load-time address-binding schemes; logical (virtual) and physical addresses differ in execution-time address-binding scheme

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 13

CSC312: Operating System II

- o The set of all logical addresses generated by a program is a logical address space.
- o The set of all physical addresses corresponding to these logical addresses is a physical address space.

Program issues addresses in a logical/virtual address space which must be translated to physical address space

- o Think of the program as having a contiguous logical/virtual address space that starts at 0,
- o and a contiguous physical address space that starts somewhere else

Thus, in the execution-time address-binding scheme, the logical and physical address spaces differ.

- o The run-time mapping from virtual to physical addresses is done by a hardware device in the CPU called the memory-management unit (MMU).

The MMU has two special registers that are accessed by the CPU's control unit.

- o A data to be sent to main memory or retrieved from memory is stored in the Memory Data Register (MDR).
- o The desired logical memory address is stored in the Memory Address Register (MAR).

In MMU scheme, the value in the relocation register (i.e. the base register) is added

to every address generated by a user process at the time it is sent to memory.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 14

CSC312: Operating System II

□ For example, if the base is at 14000, then an attempt by the user to address location 0 is dynamically relocated to location 14000; an access to location 346 is mapped to location 14346.

□ We now have two different types of addresses:

- o logical addresses (in the range 0 to max) and
- o physical addresses (in the range $R + 0$ to $R + \text{max}$ for a base value R).

1.2.4 Dynamic Loading

With dynamic loading, a routine is not loaded until it is called.

- o All routines are kept on disk in a re-locatable load format.
- o The main program is loaded into memory and is executed.

When a routine needs to call another routine, the calling routine first checks to see whether the other routine has been loaded or not

- o If it has not, the re-locatable linking loader is called to load the desired routine into memory and to update the program's address tables to reflect this change. Then control is passed to the newly loaded routine.

Dynamic loading does not require special support from the operating system.

- o It is the responsibility of the users to design their programs to take advantage of such a method.
- o Operating systems may help the programmer, however, by providing library routines to implement dynamic loading.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 15

CSC312: Operating System II

The advantage of dynamic loading is that a routine is loaded only when it is needed. This method is particularly useful when large amounts of code are needed to handle infrequently occurring cases, such as error routines.

- o In this case, although the total program size may be large, the portion that is used (and hence loaded) may be much smaller.

1.2.5 Dynamic Linking and Shared Libraries

Dynamically linked libraries are system libraries that are linked to user programs when the programs are run.

- o A dynamic link library (DLL) is a collection of small programs that can be loaded when needed by larger programs and used at the same time.
- o The small program lets the larger program communicate with a specific device, such as a printer or scanner.
- o It is often packaged as a DLL program, which is usually referred to as a DLL file.

One of the main tasks for linker is to make code of library functions (eg printf(),

scanf(), sqrt(), ..etc) available to user program.

Some operating systems support only static linking, in which system libraries are treated like any other object module and are combined by the loader into the binary program image (e.g. .lib files in Windows)

Dynamic linking, in contrast, is similar to dynamic loading. Here, though, linking, rather than loading, is postponed until execution time.

- o This feature is usually used with system libraries, such as language subroutine libraries.

- o With dynamic linking, a stub is included in the image for each library routine reference.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 16

CSC312: Operating System II

□ The stub is a small piece of code that indicates how to locate the appropriate memory-resident library routine or how to load the library if the routine is not already present.

Unlike dynamic loading, dynamic linking and shared libraries generally require help from the operating system.

- o If the processes in memory are protected from one another, then the operating system is the only entity that can check to see whether the needed routine is in another process's memory space or that can allow multiple processes to access the same memory addresses.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 17

CSC312: Operating System II

1.3 Methods of Memory Management

Memory management methods include:

- i. Swapping
- ii. Partitioning
- iii. Segmentation
- iv. Paging, and
- v. Virtual memory

1.3.1 Swapping

A process needs to be in the memory to be executed. What if there is no enough memory for all processes? A process, however, can be swapped temporarily out of memory to a backing store, and then brought back into memory for continued execution.

- o Backing store is a fast disk large enough to accommodate copies of all memory images for all users; it must provide direct access to these memory images.

- o Swapping makes it possible for the total physical address space of all

processes to exceed the real physical memory of the system, thus increasing the degree of multiprogramming in a system.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 18

CSC312: Operating System II

The system maintains a ready queue of all processes whose memory images are on the backing store or in memory and are ready to run.

Whenever the CPU scheduler decides to execute a process, it calls the dispatcher.

- o The dispatcher checks to see whether the next process in the queue is in memory.
- o If it is not, and if there is no free memory region, the dispatcher swaps out a process currently in memory and swaps in the desired process.
- o It then reloads registers and transfers control to the selected process.

The context-switch time in such a swapping system is fairly high.

o To get an idea of the context-switch time, let's assume that the user process is 100 MB in size and the backing store is a standard hard disk with a transfer rate of 50 MB per second.

- o The actual transfer of the 100-MB process to or from main memory takes $100 \text{ MB} / 50 \text{ MB per second} = 2 \text{ seconds}$

The swap time is 2000 milliseconds (i.e. 2000ms)

- o Since we must swap both out and in, the total swap time is about 4,000 milliseconds.

The total transfer time is directly proportional to the amount of memory swapped.

- o If we have a computer system with 4 GB of main memory and a resident operating system taking 1 GB, the maximum size of the user process is 3 GB.

To reduce the cost of memory swap time,

- o by knowing how much memory is really being used
- o System calls to inform OS of memory use via `request_memory()` and `release_memory()`

Standard swapping is not used in modern operating systems, but modified version common

- Swap only when free memory extremely low

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 19

CSC312: Operating System II

Swapping on Mobile Systems

Although most operating systems for PCs and servers support some modified version of swapping,

- o Mobile systems typically do not support swapping in any form.
- Mobile devices generally use flash memory rather than more spacious hard disks as their persistent storage.

- o The resulting space constraint is one reason why mobile operating-system designers avoid swapping.
- o Other reasons include
 - the limited number of writes that flash memory can tolerate before it becomes unreliable and
 - the poor throughput between main memory and flash memory in these devices.

Instead other methods are used to free memory if low

- o iOS asks apps to voluntarily relinquish allocated memory
 - Read-only data thrown out and reloaded from flash if needed
 - Failure to free can result in termination
- o Android terminates apps if low free memory, but first writes application state to flash for fast restart

1.3.2 Partitioning

Single Partitioning (Contiguous Allocation)

The main memory must accommodate both operating system and the various user spaces.

The memory is usually divided into two partitions:

- one for the resident operating system and
- one for the user processes.

The operating system may be placed in the high memory or the low memory.

- o The position of the interrupt vector usually affects this decision.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 20

CSC312: Operating System II

Since the interrupt vector is often in the low memory, programmers place the OS in low memory too.

- o It is desirable to have several user processes residing in the memory at the same time.

In contiguous memory allocation, each process is contained in a single contiguous section of memory.

- o Relocation registers are used to protect user processes from each other, and from changing operating-system code and data
 - Base register contains value of smallest physical address (for example, relocation = 100040).
 - Limit register contains range of logical addresses – each logical address must be less than the limit register (for example, limit = 74600).

The MMU maps the logical address dynamically by adding the value in the relocation register. This mapped address is sent to memory as shown below

Hardware support for relocation and limit registers

Multiple Partitioning

In a uni-programming system, main memory is divided into two parts: one part for the operating system (resident monitor, kernel) and one part for the program currently being executed. In a multi-programming system, the “user” part of memory must be further subdivided to accommodate multiple processes.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 21

CSC312: Operating System II

There are variations of achieving this multiple partitioning. These include:

- i. Fixed partitioning
- ii. Variable partitioning

□ Fixed Partitioning

Physical memory is broken up into fixed partitions

- o partitions may have different sizes, but partitioning never changes
- o Each partition may contain exactly one process. Thus, the degree of multiprogramming is bound by the number of partitions.
- o When a partition is free, a process is selected from the input queue and is loaded into the free partition. When the process terminates, the partition becomes available for another process.
- o This method was originally
 - used by the IBM OS/360 operating system (called MFT) but is no longer in use.

There are variations of fixed partitioning:

- i. Equal-size partitions
 - o Any process whose size is less than or equal to the partition size can be loaded into an available partition
 - o The operating system can swap out a process if all partitions are full and no process is in the Ready or Running state

Disadvantages of equal-size partitioning

- A program may be too big to fit in a partition
- Main memory utilization is inefficient.
- o Any program, regardless of size, occupies an entire partition
- o Internal fragmentation. Waste space due to the block of data loaded being smaller than the partition

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 22

CSC312: Operating System II

- ii. Unequal-size partitioning

Assign each process the smallest partition to which it will fit

Advantages:

- Process are always assigned in such a way as to minimize wasted memory

within a partition → internal fragmentation

□ Relatively simple and require minimal OS software and overhead

Disadvantages:

□ Limitations on the active number of processes, number of partitions specified at system generation time

□ Small jobs cannot utilize partition space efficiently; In most cases it is an inefficient technique

Memory assignment for fixed partitioning

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 23

CSC312: Operating System II

Placement Algorithm for fixed partition

◆ Equal-size partitions

o Because all partitions are of equal size, it does not matter which partition is used

◆ Unequal-size partitions

o Can assign each process to the smallest partition within which it will fit

o Queue for each partition size

o Processes are assigned in such a way as to minimize wasted memory within a partition

Variable/Dynamic Partitioning

Physical memory is broken up into partitions dynamically – partitions are tailored to programs. Partitions are of variable length and number

o Process is allocated exactly as much memory as it requires

o When a process arrives, it is allocated memory from a hole large enough to accommodate it

□ A Hole is a block of available memory; holes of various sizes are scattered throughout memory

o The operating system keeps a table indicating which parts of memory are available and which are occupied.

o This technique was used by IBM's mainframe operating system, OS/MVT

Disadvantages of Dynamic partitioning

o Memory becomes more and more fragmented with time

o Memory utilization declines

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 24

CSC312: Operating System II

Effect of dynamic partitioning

Placement Algorithms for Dynamic Partitioning

First fit: Allocate the first hole that is big enough.

o Searching can start either at the beginning of the set of holes or at the

location where the previous first-fit search ended.

- o We can stop searching as soon as we find a free hole that is large enough.
- o The idea is to minimize the time to analyze the memory space available

Best fit: Allocate the smallest hole that is big enough.

- o One must search the entire list, unless the list is ordered by size.
- o This strategy produces the smallest leftover hole.
- o The Idea is to minimize wastage of free memory space

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 25

CSC312: Operating System II

Worst fit: Allocate the largest hole.

- o Again, one must search the entire list, unless it is sorted by size.
- o This strategy produces the largest leftover hole, which may be more useful than the smaller leftover hole from a best-fit approach.
- o The idea is to increase the possibility that another process can use the left-over space

The example below illustrate the allocation of 12kb to a process using the three algorithms

Fragmentation

The allocation and de-allocation of memory creates a condition called fragmentation. This means that there are lots of small fragments (or holes) of free memory that cannot be used by any other process. This results in a reduction in the amount of total available memory.

■ External Fragmentation – Gaps between allocated contiguous memory - total memory space exists to satisfy a request, but it is not contiguous

- o Dynamic partitioning problem

■ Internal Fragmentation – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used

- o Fixed partitioning problem

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 26

CSC312: Operating System II

How to solve fragmentation problem

Reduce external fragmentation by compaction

□ Shuffle memory contents to place all free memory together in one large block

- o Swap a program out
- o Re-load it, adjacent to another
- o Adjust its base register

□ Compaction is possible only if relocation is dynamic, and is done at execution time

Exercise

Given free memory partitions of 100K, 500K, 200K, 300K, and 600K (in order),

- i. How would each of the First-fit, Best-fit, and Worst-fit algorithms place processes of 212K, 417K, 112K, and 426K (in order)?
- ii. Which algorithm makes the most efficient use of memory?

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 27

CSC312: Operating System II

1.3.3 Paging

Paging is a technique of memory management in which physical memory is broken into blocks of the same size called pages.

The basic method for implementing paging involves breaking

- o physical memory into fixed-sized blocks called frames and
- o logical memory into blocks of the same size called pages.

When a process is to be executed, its corresponding pages are loaded into any available memory frames.

- A frame is a fixed-length block of main memory
- o Operating system keeps track of all free frames.
- o Operating system needs n free frames to run a program of size n pages.

The hardware support for paging is illustrated in the diagram below. Every address generated by the CPU is divided into two parts:

- o Page number (p) -- page number is used as an index into a page table which contains base address of each page in physical memory.
- o Page offset (d) -- page offset is combined with base address to define the physical memory address.

Paging Hardware

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 28

CSC312: Operating System II

The paging model of memory is shown in the following diagram

Paging model of logical and physical memory

The page size (like the frame size) is defined by the hardware. The size of a page is a power of 2, varying between 512 bytes and 1 GB per page, depending on the computer architecture.

- o The selection of a power of 2 as a page size makes the translation of a logical address into a page number and page offset particularly easy.
- o If the size of the logical address space is 2^m , and a page size is 2^n bytes, then the high-order $m - n$ bits of a logical address designate the page number, and the n low-order bits designate the page offset.

Thus, the logical address is as follows:

where p is an index into the page table and d is the displacement within the page.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 29

CSC312: Operating System II

As a concrete example, consider the memory in the diagram below. Here, in the logical address, $n = 2$ and $m = 4$. Using a page size of 4 bytes and a physical memory of 32 bytes (8 pages), we show how the programmer's view of memory can be mapped into physical memory.

Logical address 0 is page 0, offset 0.

o Indexing into the page table, we find that page 0 is in frame 5.

□ Thus, logical address 0 maps to physical address 20 $[= (5 \times 4) + 0]$.

o Logical address 3 (page 0, offset 3) maps to physical address 23 $[= (5 \times 4) + 3]$.

o Logical address 4 is page 1, offset 0; according to the page table, page 1 is mapped to frame 6.

□ Thus, logical address 4 maps to physical address 24 $[= (6 \times 4) + 0]$.

o Logical address 13 maps to physical address 9.

Paging example for a 32-byte memory with 4-byte pages

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 30

CSC312: Operating System II

Advantages of Paging

- o Easy to allocate: keep a free list of available pages and grab the first one.
- o Easy to swap since everything is the same size, which is usually the same size as disk blocks to and from which pages are swapped.
- o External fragmentation is avoided by using paging technique
- any free frame can be allocated to a process that needs it.

Disadvantages of Paging

- o Efficiency of access: even small page tables are generally too large to load into fast memory in the relocation box. Instead, page tables are kept in main memory and the relocation box only has the page table's base address. It thus takes one overhead reference for every real memory reference.
- o Internal fragmentation: page size doesn't match up with information size. The larger the page, the worse this is. But no external fragmentation.

1.3.4 Segmentation

Segmentation supports the user-view of memory that the logical address space becomes a collection of (typically disjoint) segments.

- o A segment is a variable-length block of data that resides in secondary memory
- o Segments have a name (or a number) and a length.
- o Addresses specify segment, and offset within segment

- o To access memory, user program specifies segment + offset

- Logical address = segment name (number) + offset

Segmentation permits the physical address space of a process to be non-contiguous.

Typical segments include

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 31

CSC312: Operating System II

- o global variables

- o procedure call stack

- o code for each function

- o local variables for each large data structures

Segmentation hardware

In the segmentation hardware above, each entry in the segment table has a segment base and a segment limit.

- o The segment base contains the starting physical address where the segment resides in memory, and the segment limit specifies the length of the segment.

- o A logical address consists of two parts: a segment number, and an offset into that segment

- The segment number is used as an index to the segment table.

- The offset of the logical address must be between 0 and the segment limit. If it is not, we trap to the operating system (logical addressing attempt beyond end of segment).

- o When an offset is legal, it is added to the segment base to produce the address in physical memory of the desired byte.

- o The segment table is thus essentially an array of base-limit register pairs

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 32

CSC312: Operating System II

Example to illustrate Segmentation:

We have five segments numbered from 0 through 4. The segments are stored in physical memory as shown in the diagram below.

The segment table has a separate entry for each segment, giving the beginning address of the segment in physical memory (or base) and the length of that segment (or limit).

For example,

- segment 2 is 400 bytes long and begins at location 4300. Thus, a reference to byte 53 of segment 2 is mapped onto location $4300 + 53 = 4353$.

- A reference to segment 3, byte 852, is mapped to 3200 (the base of segment 3) + 852 = 4052.

- A reference to byte 1222 of segment 0 would result in a trap to the operating system, as this segment is only 1,000 bytes long.

Example of segmentation

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 33

CSC312: Operating System II

Advantages:

- o Segments can be shared, swapped, and assigned to storage independently.
- o Segment has defined length so it is possible to detect access outside of the segment.

□ It throws new type of error – segmentation fault

Disadvantages

- o It is not easy to place the segment into memory
- Segments has different size
- Memory fragmentation
- Segment moving has big overhead (one comparison, one addition)

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 34

CSC312: Operating System II

Exercises

1. Consider the following segment table:

Segment Base Length

0 219 600

1 2300 14

2 90 100

3 1327 580

4 1952 96

What are the physical addresses for the following logical addresses?

- a. 0,430
- b. 1,10
- c. 2,500
- d. 3,400
- e. 4,112

2. Assuming a 1-KB page size, what are the page numbers and offsets for the following address references (provided as decimal numbers):

- a. 3085
- b. 42095
- c. 215201
- d. 650000
- e. 2000001

3. Although Android does not support swapping on its boot disk, it is possible to set up a swap space using a separate SD non-volatile memory card. Why would Android disallow swapping on its boot disk yet allow it on a secondary disk?

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 35

CSC312: Operating System II

1.3.5 Virtual Memory

Despite the fact that most modern computers have between 1 GB and 4 GB of RAM, it is possible to run out of memory. This can happen when too many processes are running at the same time, or when a process requires more memory than is currently available (unused). One solution to this problem is to use virtual memory.

- o Virtual memory is a technique that allows the execution of processes that are not completely in memory

This means that the operating system's memory manager will use a portion of another storage device to act as if it is extra RAM.

- o In most cases, this virtual memory space will be on a permanent storage device such as a hard disk.

- However, some newer operating systems (such as Windows Vista and Windows 7) also allow one to use a removable media device like a USB flash drive as virtual memory (called Windows Ready Boost).

- o The memory manager will manage this resource so that it gives the illusion that the computer has more RAM than is actually installed.

Advantages

- o One major advantage of this scheme is that programs can be larger than physical memory.
- o Further, virtual memory abstracts main memory into an extremely large, uniform array of storage, separating logical memory as viewed by the user from physical memory.
- o This technique frees programmers from the concerns of memory-storage limitations.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 36

CSC312: Operating System II

Disadvantages

- o Virtual memory is not easy to implement
- o May substantially decrease performance if it is used carelessly.

Implementation of Virtual Memory

One common technique for implementing virtual memory is demand paging.

- o With demand-paged virtual memory, pages are loaded only when they are demanded during program execution.
- o Pages that are never accessed are thus never loaded into physical memory.

A demand-paging system is similar to a paging system with swapping (see diagram below) where processes reside in secondary memory (usually a disk). When a

process is to be executed, it is swapped into memory. Rather than swapping the entire process into memory, though, a pager (popularly called lazy swapper) is used.

- o A pager is concerned with swapping the individual pages of a process that will be needed.

- o It never swaps a page into memory unless that page will be needed.

Transfer of a paged memory to contiguous disk space.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 37

CSC312: Operating System II

When a process is to be swapped in, the pager guesses which pages will be used before the process is swapped out again

- o Instead of swapping in a whole process, the pager brings only those pages into memory.

- o Thus, it avoids reading into memory pages that will not be used anyway, decreasing the swap time and the amount of physical memory needed.

With this scheme, some forms of hardware support are needed to distinguish between the pages that are in memory and the pages that are on the disk.

The valid-invalid bit scheme is used for this purpose.

- o When this bit is set to “valid,” the associated page is both legal and in memory.

- o If the bit is set to “invalid,” the page either is not valid (that is, not in the logical address space of the process) or is valid but is currently on the disk.

The page-table entry for a page that is brought into memory is set as usual, but the page-table entry for a page that is not currently in memory is either simply marked invalid or contains the address of the page on disk. This situation is depicted in the diagram below

Page table when some pages are not in main memory

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 38

CSC312: Operating System II

But what happens if the process tries to access a page that was not brought into memory?

- o Access to a page marked invalid causes a page fault.

The paging hardware, in translating the address through the page table, will notice that the invalid bit is set, causing a trap to the operating system.

- o This trap is the result of the operating system’s failure to bring the desired page into memory.

The procedure for handling this page fault is straightforward (See diagram below):

- We check an internal table (usually kept with the process control block) for this process to determine whether the reference was a valid or an invalid memory access.

- If the reference was invalid, we terminate the process. If it was valid but we have not yet brought in that page, we now page it in.
- We find a free frame (by taking one from the free-frame list, for example).
- We schedule a disk operation to read the desired page into the newly allocated frame.
- When the disk read is complete, we modify the internal table kept with the process and the page table to indicate that the page is now in memory.
- We restart the instruction that was interrupted by the trap. The process can now access the page as though it had always been in memory.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 39

CSC312: Operating System II

Steps in handling a page fault

Page replacement

While a user process is executing, a page fault occurs. The operating system determines where the desired page is residing on the disk but then finds that there are no free frames on the free-frame list; all memory is in use.

The operating system has several options at this point.

- o It could terminate the user process
- o The operating system could instead swap out a process, freeing all its frames and reducing the level of multiprogramming.
- o Find one that is not currently being used and free it

□ This is the idea of Page replacement

Page replacement takes the following approach.

- o If no frame is free, we find one that is not currently being used and free it.
- o We can free a frame by writing its contents to swap space and changing the page table (and all other tables) to indicate that the page is no longer in memory

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 40

CSC312: Operating System II

- o We can now use the freed frame to hold the page for which the process faulted.

The above steps are modified as follows:

1. Find the location of the desired page on the disk.
2. Find a free frame:
 - a. If there is a free frame, use it.
 - b. If there is no free frame, use a page-replacement algorithm to select a victim frame.
 - c. Write the victim frame to the disk; change the page and frame tables accordingly.
3. Read the desired page into the newly freed frame; change the page and

frame tables.

4. Continue the user process from where the page fault occurred.

Notice that, if no frames are free, two page transfers (one out and one in) are required. This situation effectively doubles the page-fault service time and increases the effective access time accordingly.

Page replacement

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 41

CSC312: Operating System II

Page replacement Algorithms

There are many different page-replacement algorithms. Every operating system probably has its own replacement scheme.

- o How do we select a particular replacement algorithm?

- In general, we want the one with the lowest page-fault rate.

We evaluate an algorithm by running it on a particular string of memory references and computing the number of page faults.

- o The string of memory references is called a reference string.

To determine the number of page faults for a particular reference string and page-replacement algorithm,

- o we also need to know the number of page frames available.

- o as the number of frames available increases, the number of page faults decreases.

- o adding physical memory increases the number of frames.

We next illustrate several page-replacement algorithms. In doing so, we use the reference string

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

for a memory with three frames.

FIFO Page Replacement

The simplest page-replacement algorithm is a first-in, first-out (FIFO) algorithm.

A FIFO replacement algorithm associates with each page the time when that page was brought into memory.

- o When a page must be replaced, the oldest page is chosen.

Notice that it is not strictly necessary to record the time when a page is brought in.

- o We can create a FIFO queue to hold all pages in memory.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 42

CSC312: Operating System II

- o We replace the page at the head of the queue. When a page is brought into memory, we insert it at the tail of the queue.

Illustration:

For our example reference string, our three frames are initially empty.

- o The first three references (7, 0, 1) cause page faults and are brought into these empty frames.
- o The next reference (2) replaces page 7, because page 7 was brought in first.
- o Since 0 is the next reference and 0 is already in memory, we have no fault for this reference.
- o The first reference to 3 results in replacement of page 0, since it is now first in line. Because of this replacement, the next reference, to 0, will fault.
- o Page 1 is then replaced by page 0.

This process continues as shown in the diagram below. Every time a fault occurs, we show which pages are in our three frames. There are fifteen faults altogether

FIFO page-replacement algorithm

Least Recently Used (LRU) Algorithm.

LRU replacement associates with each page the time of that page's last use. When a page must be replaced,

- o LRU chooses the page that has not been used for the longest period of time.

The result of applying LRU replacement to our example reference string is shown in the diagram below. The LRU algorithm produces twelve faults.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 43

CSC312: Operating System II

LRU page-replacement algorithm

Other replacement algorithms include

- i. Second Chance
- ii. Enhanced Second Chance
- iii. Least Frequently Used (LFU)
- iv. Most Frequently Used (MFU)
- v. Stack algorithm

These algorithms are left as research topics for students.

Hit Ratios: Determining Which Replacement Policy to Use

Using each of the replacement policies to swap pages between main memory and the page file will result in different hit ratios.

- o A hit ratio is the number of times that a page is actually found in main memory, as opposed to the number of page faults generated (requiring the memory manager to retrieve the page from virtual memory).
- o To calculate the hit ratio, we divide the number of non page faults by the total number of page requests (the number of times that data has been sent between the CPU and main memory).
- o Page faults occur when a page is requested by the CPU that is not currently in the page table (in other words, not currently in RAM).

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 44

CSC312: Operating System II

Calculating Hit Ratios

To understand how to calculate hit ratios, we will examine an example that uses a RAM space of three (3) frames, and the following processing sequence:

12131415232415

In these examples, Y = a hit (the page is found in RAM), and N = a page fault (the page must be retrieved from virtual memory)

FIFO Algorithm

LRU Algorithm

Thrashing

Thrashing is a condition where a process is busy swapping pages in and out due to insufficient frames in physical memory.

- o A process is thrashing if it is spending more time paging than executing user instructions.

- o A process is swapped out just before it is needed

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 45

CSC312: Operating System II

Thrashing results in severe performance problems:

- o Low CPU utilization
- o High disk utilization
- o Utilization of other I/O devices

Causes of Thrashing

Thrashing is typically caused by too many processes running at the same time. All the processes will compete for the limited amount of physical memory installed on the computer.

- o If the computer is thrashing, applications may stop responding (or run very slowly).

How to Prevent Thrashing

- The current best practice in implementing a computer facility is to include enough physical memory, whenever possible, to avoid thrashing and swapping.

- o From smart phones through mainframes, providing enough memory to keep all working sets in memory concurrently, except under extreme conditions, gives the best user experience.

- We can limit the effects of thrashing by using a local replacement algorithm (or priority replacement algorithm).

- o With local replacement, if one process starts thrashing, it cannot steal frames from another process and cause the latter to thrash as well.

However, the problem is not entirely solved.

- To prevent thrashing, we must provide a process with as many frames as it

needs.

o One technique to know the number of frames a process needs is the working-set model.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 46

CSC312: Operating System II

o The working-set strategy starts by looking at how many frames a process is actually using.

□ If a page is in active use, it will be in the working set. If it is no longer being used, it will drop from the working set

Exercises

1. Under what circumstances do page faults occur? Describe the actions taken by the operating system when a page fault occurs.

2. Consider the following page reference string:

1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6

How many page faults would occur for the following replacement algorithms, assuming one, two, three, four, five, six, and seven frames?

Remember that all frames are initially empty, so your first unique pages will cost one fault each.

i. LRU replacement

ii. FIFO replacement

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 47

CSC312: Operating System II

2. CPU SCHEDULING

A process is executed until it must wait, typically for the completion of some I/O request.

o In a simple computer system, the CPU then just sits idle

o All this waiting time is wasted; no useful work is accomplished.

CPU scheduling is the basis of multi-programmed operating systems. By switching the CPU among processes, the operating system can make the computer more productive.

o The objective of multiprogramming is to have some process running at all times, to maximize CPU utilization.

o Several processes are kept in memory at one time.

o Every time one process has to wait (perhaps for I/O request), another process can take over use of the CPU.

The CPU is, of course, one of the primary computer resources. Thus, its scheduling is central to operating-system design.

□ CPU scheduling deals with the problem of deciding which of the processes in the ready queue is to be allocated the CPU.

2.1 Basic Concepts

CPU-I/O Burst Cycle

The success of CPU scheduling depends on CPU-I/O burst cycle since process execution consists of a cycle of CPU execution and I/O wait.

- Processes alternate between these two states.

Process execution begins with a CPU burst. That is followed by an I/O burst, which is followed by another CPU burst, then another I/O burst, and so on (see diagram below)

- CPU burst – process bound to CPU

- I/O burst – process bound to I/O

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 48

CSC312: Operating System II

- CPU Burst time - amount of time the process uses the processor before it is no longer ready

Alternating sequence of CPU and I/O bursts

CPU Scheduler

Whenever the CPU becomes idle, the operating system must select one of the processes in the ready queue to be executed.

- The selection process is carried out by the short-term scheduler, or CPU scheduler.

- The scheduler selects a process from the processes in memory that are ready to execute and allocates the CPU to that process.

- Conceptually, all the processes in the ready queue are lined up waiting for a chance to run on the CPU.

Preemptive & Non-preemptive Scheduling

CPU scheduling decisions may take place when a process:

- Switches from running to waiting state (I/O request).
- Switches from running to ready state (when interrupt occurs).

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 49

CSC312: Operating System II

- Switches from waiting to ready (completion of I/O).

- Terminates

Scheduling under 1 and 4 is non-preemptive:

- Once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU either by terminating or by switching to the waiting state.

Scheduling under 2 and 3 is preemptive:

- Allows a running process to be interrupted by a high priority process

- The process remains in the ready queue until it gets the next opportunity to execute

Preemptive scheduling incurs a cost.

- o Assume that two processes sharing data. One may be in the middle of updating the data when it is preempted and the second process is running. The second process may try to read the data which are currently in an inconsistent state

Dispatcher

Another component involved in the CPU-scheduling function is the dispatcher.

□ Dispatcher module gives control of the CPU to the process selected by the short-term scheduler; this function involves:

- o switching context
- o switching to user mode
- o jumping to the proper location in the user program to restart that program

□ Dispatch latency- time it takes for the dispatcher to stop one process and start another running.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 50

CSC312: Operating System II

2.2 Scheduling Criteria & Optimization

Different CPU-scheduling algorithms have different properties and characteristics and may favor one class of processes over another. The characteristics used for comparison can make a substantial difference in the determination of the best algorithm.

The criteria include:-

- o CPU utilization – keep the CPU as busy as possible (40 % for a lightly loaded system to 90 % for heavily used system)
- o Maximize CPU utilization
- o Throughput – # of processes that complete their execution per time unit (1 process / hour for long processes, 10 process / second for short transactions)
- o Maximize throughput
- o Turnaround time – amount of time it takes to execute a particular process (from time of submission to time of completion)
- o Minimize turnaround time
- o Waiting time – amount of time a process has been waiting in the ready queue
- o Minimize waiting time
- o Response time – amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment)
- o Minimize response time

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 51

CSC312: Operating System II

2.3 Scheduling Algorithms

There are many different CPU-scheduling algorithms. These include:

- g. First-Come, First-Served Scheduling
- h. Shortest-Job-First Scheduling
- i. Priority Scheduling
- j. Round-Robin Scheduling
- k. Multi-level Queue Scheduling
- l. Multilevel feedback Queue Scheduling

First-Come, First-Served Scheduling

The simplest CPU-scheduling algorithm is the first-come, first-served (FCFS) scheduling algorithm.

- o With this scheme, the process that requests the CPU first is allocated the CPU first.
- o The implementation of the FCFS policy is easily managed with a FIFO queue.
- o The average waiting time under the FCFS policy is often quite long.

Consider the following set of processes that arrive at time 0, with the length of the CPU burst given in milliseconds:

Process Burst Time

P1 24

P2 3

P3 3

If the processes arrive in the order P1, P2, P3, and are served in FCFS order, we get the result shown in the following Gantt chart

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 52

CSC312: Operating System II

The waiting time is 0 milliseconds for process P1, 24 milliseconds for process P2, and 27 milliseconds for process P3.

Thus, the average waiting time is $(0 + 24 + 27)/3 = 17$ milliseconds.

If the processes arrive in the order P2, P3, P1, however, the results will be as shown in the following Gantt chart:

Waiting time for P1 = 6, P2 = 0, P3 = 3

Average waiting time is now $(6 + 0 + 3)/3 = 3$ milliseconds.

This reduction is substantial.

Shortest-Job-First Scheduling

Associate with each process the length of its next CPU burst. Use these lengths to schedule the process with the shortest time.

Two schemes:

- o non-preemptive – once CPU given to the process it cannot be pre-empted until completes its CPU burst.
- o preemptive – if a new process arrives with CPU burst length less than remaining time of current executing process, preempt. This scheme is known

as the Shortest-Remaining-Time-First (SRTF).

SJF is optimal – gives minimum average waiting time for a given set of processes.

As an example of SJF scheduling, consider the following set of processes, with the length of the CPU burst given in milliseconds:

Process Burst Time

P1 6

P2 8

P3 7

P4 3

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 53

CSC312: Operating System II

Using non-preemptive SJF scheduling, we would schedule these processes according to the following Gantt chart:

Waiting time: P1 = 3, P2 = 16, P3 = 9, P4 = 0

Average waiting time = $(3 + 16 + 9 + 0)/4 = 7\text{ms}$

By comparison, if we were using the FCFS scheduling scheme, the average waiting time would be 10.25 ms

Consider another example using both preemptive and non-preemptive SJF

Process Arrival Time Burst Time

P1 0 8

P2 1 4

P3 2 9

P4 3 5

Preemptive SJF:

Average waiting time : P1 : $10 - 1 = 9$

P2 : $1 - 1 = 0$

P3 : $17 - 2 = 15$

P4 : $5 - 3 = 2$

AWT = $(9+0+15+2) / 4 = 6.5\text{ ms}$

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 54

CSC312: Operating System II

Non-Preemptive SJF:

AWT = $0 + (8 - 1) + (12 - 3) + (17 - 2) / 4 = 7.75\text{ ms}$

Priority Scheduling

The SJF algorithm is a special case of the general priority-scheduling algorithm. A priority is associated with each process, and the CPU is allocated to the process with the highest priority.

o Equal-priority processes are scheduled in FCFS order.

o An SJF algorithm is simply a priority algorithm where the priority (p) is the

inverse of the (predicted) next CPU burst. The larger the CPU burst, the lower the priority, and vice versa.

As an example, consider the following set of processes, assumed to have arrived at time 0 in the order P1, P2, ..., P5, with the length of the CPU burst given in milliseconds:

Process Burst Time Priority

P1 10 3

P2 1 1

P3 2 4

P4 1 5

P5 5 2

Using priority scheduling, we would schedule these processes according to the following Gantt chart:

$$\text{AWT} = (0 + 1 + 6 + 16 + 18)/5 = 41/5 \\ = 8.2\text{ms}$$

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 55

CSC312: Operating System II

Priority scheduling can be either preemptive or non-preemptive. When a process arrives at the ready queue, its priority is compared with the priority of the currently running process.

□ A preemptive priority scheduling algorithm will preempt the CPU if the priority of the newly arrived process is higher than the priority of the currently running process.

□ A non-preemptive priority scheduling algorithm will simply put the new process at the head of the ready queue.

A major problem with priority scheduling algorithms is indefinite blocking, or starvation.

o A process that is ready to run but waiting for the CPU can be considered blocked.

o A priority scheduling algorithm can leave some low priority processes waiting indefinitely

Round-Robin Scheduling

The round-robin (RR) scheduling algorithm is designed especially for timesharing systems. It is similar to FCFS scheduling, but preemption is added to switch between processes. A small unit of time, called a time quantum (or time slice), is defined.

o The ready queue is treated as a circular queue.

o The CPU scheduler goes around the ready queue, allocating the CPU to each process for a time interval of up to 1 time quantum.

The average waiting time under the RR policy is often long. Consider the following set of processes that arrive at time 0, with the length of the CPU burst given in

milliseconds:

Process Burst Time

P1 24

P2 3

P3 3

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 56

CSC312: Operating System II

Using Time quantum = 4 ms

The Gantt chart is shown as follows:

Waiting time for P1 = 26 - 20 = 6

P2 = 4

P3 = 7

AWT = $(6+4+7) / 3 = 5.66$ ms

The performance of the RR algorithm depends heavily on the size of the time-quantum.

- o If time-quantum is very large (infinite) then RR policy is same as FCFS policy.

- o If time quantum is very small, RR approach is called processor sharing and appears to the users as though each of n process has its own processor running at 1/n the speed of real processor.

Multilevel Queue Scheduling

A multilevel queue-scheduling algorithm partitions the ready queue into several separate queues.

- o Foreground (interactive)

- o Background (batch)

- o The foreground (interactive) queue may have absolute priority over the background (batch) queue

These two types of processes have different response-time requirements and so may have different scheduling needs. Each queue has its own scheduling algorithm

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 57

CSC312: Operating System II

- o foreground queue - RR

- o background queue - FCFS

The processes are permanently assigned to one queue, generally based on some property of the process, such as

- o memory size,

- o process priority, or

- o process type.

Scheduling must be done between the queues

- o Fixed priority scheduling; (i.e., serve all from foreground then from background).

- Danger of starvation.

- o Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR, 20% to background in FCFS

Example of a multilevel queue scheduling algorithm with five queues

1. System processes
2. Interactive processes
3. Interactive editing processes
4. Batch processes
5. Student processes

Multilevel queue scheduling

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 58

CSC312: Operating System II

Each queue has absolute priority over lower-priority queue.

No process in the batch queue, for example, could run unless the queues for system processes, interactive processes, and interactive editing processes were all empty.

- o If an interactive editing process entered the ready queue while a batch process was running, the batch process would be preempted.

Multilevel Feedback Queue Scheduling

Multilevel feedback queue scheduling allows a process to move between queues.

The idea is to separate processes with different CPU-burst characteristics.

- o If a process uses too much CPU time, it will be moved to a lower-priority queue.

- o This scheme leaves I/O-bound and interactive processes in the higher-priority queues.

- o Similarly, a process that waits too long in a lower priority queue may be moved to a higher-priority queue.

- o This form of aging prevents starvation.

For example, consider a multilevel feedback queue scheduler with three queues, numbered from 0 to 2

Three queues:

Q0 – RR with time quantum 8 milliseconds

Q1 – RR time quantum 16 milliseconds

Q2 – FCFS

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 59

CSC312: Operating System II

Multilevel feedback queues.

Scheduling:

- o A new job enters queue Q0. When it gains CPU, job receives 8 milliseconds.

If it exhausts 8 milliseconds and does not complete, job is moved to queue Q1.

- o At Q1 the job receives 16 additional milliseconds. If it still does not complete, it is preempted and moved to queue Q2.

The scheduler first executes all processes in queue 0. Only when queue 0 is empty will it execute processes in queue 1.

- o Similarly, processes in queue 2 will be executed only if queues 0 and 1 are empty.

- o A process that arrives for queue 1 will preempt a process in queue 2.

- o A process that arrives for queue 0 will, in turn, preempt a process in queue 1.

A multilevel feedback queue scheduler is defined by the following parameters:

- o The number of queues

- o The scheduling algorithm for each queue

- o The method used to determine when to upgrade a process to a higher priority queue

- o The method used to determine when to demote a process to a lower-priority queue

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 60

CSC312: Operating System II

- o The method used to determine which queue a process will enter when that process needs service

Exercises

1. Explain the difference between preemptive and non-preemptive scheduling.

2. Suppose that the following processes arrive for execution at the times indicated. Each process will run for the amount of time listed. In answering the questions, use non-preemptive scheduling, and base all decisions on the information you have at the time the decision must be made.

Process Arrival Time Burst Time

P1 0.0 8

P2 0.4 4

P3 1.0 1

- a. What is the average turnaround time for these processes with the FCFS scheduling algorithm?

- b. What is the average turnaround time for these processes with the SJF scheduling algorithm?

3. Why is it important for the scheduler to distinguish I/O-bound programs from CPU-bound programs?

4. Consider the following set of processes, with the length of the CPU burst

given in milliseconds:

Process Burst Time Priority

P1 2 2

P2 1 1

P3 8 4

P4 4 2

P5 5 3

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 61

CSC312: Operating System II

The processes are assumed to have arrived in the order P1, P2, P3, P4, P5, all at time 0.

- Draw four Gantt charts that illustrate the execution of these processes using the following scheduling algorithms: FCFS, SJF, nonpreemptive priority (a larger priority number implies a higher priority), and RR (quantum = 2).
- What is the turnaround time of each process for each of the scheduling algorithms in part a?
- What is the waiting time of each process for each of these scheduling algorithms?
- Which of the algorithms results in the minimum average waiting time (over all processes)?

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 62

CSC312: Operating System II

3. Mutual Exclusion & Synchronization

3.1 Race Condition, Critical Section, and Mutual Exclusion

Processes frequently need to communicate with other processes. A cooperating process is one that can affect or be affected by other processes executing in the system. Cooperating processes can either directly share a logical address space (that is, both code and data) or be allowed to share data only through files or messages. However, sometimes a process has to access shared memory or files, or do other critical things that can lead to race condition

o A race condition is a situation in which multiple threads or processes read and write a shared data item and the final result depends on the relative timing of their execution

A race condition occurs when several processes access and manipulate the same data concurrently and the outcome of the execution depends on the particular order in which the access takes place. That part of the program where the shared memory is accessed is called the critical region or critical section.

o A critical section is a section of code within a process that requires access to shared resources and that must not be executed while another process is in a corresponding section of code

□ Each process must request permission to enter its

critical section. The section of code implementing this request is the entry section. The critical section may be followed by an exit section. The remaining code is the remainder section.

The general structure is illustrated below

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 63

CSC312: Operating System II

The key to preventing trouble here and in many other situations involving shared memory, shared files, and shared everything else is to find some way to prohibit more than one process from reading and writing the shared data at the same time. Put in other words, what is needed is mutual exclusion, that is, some way of making sure that if one process is using a shared variable or file, the other processes will be excluded from doing the same thing.

o Mutual exclusion is a requirement that when one process is in a critical section that accesses shared resources, no other process may be in a critical section that accesses any of those shared resources

The critical-section problem is to design a protocol that the processes can use to cooperate. Each process must request permission to enter its critical section.

We need four conditions to hold to have a good solution:

No two processes may be simultaneously inside their critical regions.

No assumptions may be made about speeds or the number of CPUs.

No process running outside its critical region may block any process.

No process should have to wait forever to enter its critical region.

Figure 3.1 below illustrates how mutual exclusion can be achieved using critical section. Here process A enters its critical region at time T1. A little later, at time T2 process B attempts to enter its critical region but fails because another process is already in its critical region and can allow only one at a time. Consequently, B is temporarily suspended until time T3 when A leaves its critical region, allowing B to enter immediately. Eventually B leaves (at T4) and it is back to the original situation with no processes in their critical regions.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 64

CSC312: Operating System II

Fig3.1 Mutual exclusion using critical regions

3.2 Solutions to Critical Section Problem

A solution to the critical-section problem must satisfy the following three requirements:

□ Mutual exclusion. If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.

□ Progress. If no process is executing in its critical section and some

processes wish to enter their critical sections, then only those processes that are not executing in their remainder sections can participate in deciding which will enter its critical section next, and this selection cannot be postponed indefinitely.

□ Bounded waiting. There exists a bound, or limit, on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 65

CSC312: Operating System II

Two general approaches are used to handle critical sections in operating systems: preemptive kernels and non-preemptive kernels.

□ A preemptive kernel allows a process to be preempted while it is running in kernel mode.

o Preemptive kernels must be carefully designed to ensure that shared kernel data are free from race conditions.

□ A non-preemptive kernel does not allow a process running in kernel mode to be preempted; a kernel-mode process will run until it exits kernel mode, blocks, or voluntarily yields control of the CPU.

o Obviously, a non-preemptive kernel is essentially free from race conditions on kernel data structures, as only one process is active in the kernel at a time

3.2.1 Peterson's Solution

Peterson solution is a classic software-based solution to the critical-section problem. It provides a good algorithmic description of solving the critical-section problem.

□ Peterson's solution is restricted to two processes that alternate execution between their critical sections and remainder sections

Fig. 3.2.The structure of process P_i in Peterson solution

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 66

CSC312: Operating System II

The variable `turn` indicates whose turn it is to enter its critical section. That is, if `turn == i`, then process P_i is allowed to execute in its critical section. The `flag` array is used to indicate if a process is ready to enter its critical section. For example, if `flag[i]` is true, this value indicates that P_i is ready to enter its critical section. To enter the critical section, process P_i first sets `flag[i]` to be true and then sets `turn` to the value `j`, thereby asserting that if the other process wishes to enter the critical section, it can do so.

o If both processes try to enter at the same time, `turn` will be set to both `i` and `j` at roughly the same time.

- o Only one of these assignments will last; the other will occur but will be overwritten immediately.
- o The eventual value of turn determines which of the two processes is allowed to enter its critical section first.

Because of the way modern computer architectures perform basic machine-language instructions, such as load and store, there are no guarantees that Peterson's solution will work correctly on such architectures

3.2.2 Mutex Locks

Operating-systems designers build software tools to solve the critical-section problem. The simplest of these tools is the mutex lock. (In fact, the term mutex is short for mutual exclusion.)

A mutex is a shared variable that can be in one of two states: unlocked or locked. Consequently, only 1 bit is required to represent it, but in practice an integer often is used, with 0 meaning unlocked and all other values meaning locked

The mutex lock is used to protect critical regions and thus prevent race conditions.

- o That is, a process must acquire the lock before entering a critical section; it releases the lock when it exits the critical section.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 67

CSC312: Operating System II

- o The acquire() function acquires the lock, and the release() function releases the lock.

A mutex lock has a boolean variable available whose value indicates if the lock is available or not. If the lock is available, a call to acquire() succeeds, and the lock is then considered unavailable. A process that attempts to acquire an unavailable lock is blocked until the lock is released.

The definition of acquire() is as follows:

Figure 3.3 illustrates the Solution to the critical-section problem using mutex locks.

Fig 3.3 Solution to the critical-section problem using Mutex locks

The definition of release() is as follows:

```
release() {
    available = true;
```

The main disadvantage of the implementation given here is that it requires busy waiting.

- o While a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the call to acquire().

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 68

CSC312: Operating System II

- o In fact, this type of mutex lock is also called a spinlock because the process “spins” while waiting for the lock to become available.
- o This continual looping is clearly a problem in a real multiprogramming system, where a single CPU is shared among many processes.
- o Busy waiting wastes CPU cycles that some other process might be able to use productively

3.2.3 Semaphores

A semaphore is another synchronization tool used by processes to synchronize their activities. A semaphore is an integer variable used for signaling among processes. It is accessed only through two standard atomic operations: wait() and signal() or sleep and wakeup as used in some text.

The definition of wait() is as follows:

```
// execute to use a resource
wait(S) {
while (S <= 0)
// busy wait
S--;
```

The definition of signal() is as follows:

```
// execute to release a resource
signal(S) {
S++;
```

All modifications to the integer value of the semaphore in the wait() and signal() operations must be executed indivisibly.

- o That is, when one process modifies the semaphore value, no other process can simultaneously modify that same semaphore value.
- o . In addition, in the case of wait(S), the testing of the integer value of S ($S \leq 0$), as well as its possible modification (S--), must be executed without interruption

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 69

CSC312: Operating System II

□ Binary & Counting Semaphore

Operating systems often distinguish between counting and binary semaphores.

The value of a binary semaphore can range only between 0 and 1. Thus, binary semaphores behave similarly to mutex locks. In fact, on systems that do not provide mutex locks, binary semaphores can be used instead for providing mutual exclusion.

The value of a counting semaphore can range over an unrestricted domain.

Counting semaphores can be used to control access to a given resource consisting of a finite number of instances. The semaphore is initialized to the number of resources available. Each process that wishes to use a resource performs a wait() operation on

the semaphore (thereby decrementing the count).

When a process releases a resource, it performs a `signal()` operation (incrementing the count). When the count for the semaphore goes to 0, all resources are being used. After that, processes that wish to use a resource will block until the count becomes greater than 0.

3.2.3.1 Semaphore Implementation with Blocking

The definitions of the `wait()` and `signal()` semaphore operations just described suffers the same problem of busy waiting just as in mutex locks.

To overcome the need for busy waiting, the definition of the `wait()` and `signal()` operations can be modified as follows:

- o When a process executes the `wait()` operation and finds that the semaphore value is not positive, it must wait.
- o However, rather than engaging in busy waiting, the process can block itself.
- o The block operation places a process into a waiting queue associated with the semaphore, and the state of the process is switched to the waiting state.
- o Then control is transferred to the CPU scheduler, which selects another process to execute.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 70

CSC312: Operating System II

- o A process that is blocked, waiting on a semaphore `S`, should be restarted when some other process executes a `signal()` operation.
- The process is restarted by a `wakeup()` operation, which changes the process from the waiting state to the ready state. The process is then placed in the ready queue

Now, the `wait()` semaphore operation can be defined as

and the `signal()` semaphore operation can be defined as

The `block()` operation suspends the process that invokes it. The `wakeup(P)` operation resumes the execution of a blocked process `P`. These two operations are provided by the operating system as basic system calls.

3.2.3.2 Problems of Implementing Semaphore with blocking

1. Deadlocks

The implementation of a semaphore with a waiting queue may result in a situation where two or more processes are waiting indefinitely for an event that can be caused only by one of the waiting processes. The event in question is the

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 71

CSC312: Operating System II

execution of a `signal()` operation. When such a state is reached, these processes are

said to be deadlocked.

- o A set of processes is in a deadlocked state when every process in the set is waiting for an event that can be caused only by another process in the set.
- o The events with which are mainly of concerned here are resource acquisition and release

Illustration of Deadlock

To illustrate this, consider a system consisting of two processes, P0 and P1, each accessing two semaphores, S and Q, set to the value 1:

Suppose that P0 executes wait(S) and then P1 executes wait(Q). When P0 executes wait(Q), it must wait until P1 executes signal(Q).

Similarly, when P1 executes wait(S), it must wait until P0 executes signal(S).

Since these signal() operations cannot be executed, P0 and P1 are deadlocked.

2. Starvation

Another problem related to deadlocks is indefinite blocking or starvation, a situation in which processes wait indefinitely within the semaphore.

- o Indefinite blocking may occur if a process is removed from the list associated with a semaphore in LIFO (last-in, first-out) order.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 72

CSC312: Operating System II

Exercises

1. Explain why spinlocks are not appropriate for single-processor systems yet are often used in multiprocessor systems.
2. Show that, if the wait() and signal() semaphore operations are not executed atomically, then mutual exclusion may be violated.
3. Illustrate how a binary semaphore can be used to implement mutual exclusion among n processes.
4. Race conditions are possible in many computer systems. Consider a banking system that maintains an account balance with two functions: deposit(amount) and withdraw(amount). These two functions are passed the amount that is to be deposited or withdrawn from the bank account balance. Assume that a husband and wife share a bank account. Concurrently, the husband calls the withdraw() function and the wife calls deposit(). Describe how a race condition is possible and what might be done to prevent the race condition from occurring.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 73

CSC312: Operating System II

4. Classical Problems of Synchronization

There are numbers of synchronization problems as examples of a large class of concurrency-control problems that are used for testing nearly every newly proposed synchronization scheme. This section presents such examples.

4.1. The Producer-Consumer Problem

The Producer-Consumer Problem (also known as bounded-buffer problem) is commonly used to illustrate the power of synchronization primitives. Two processes share a common, fixed-size buffer. One of them, the producer, puts information into the buffer, and the other one, the consumer, takes it out.

Trouble arises when the producer wants to put a new item in the buffer, but it is already full. The solution is for the producer to go to sleep, to be awakened when the consumer has removed one or more items. Similarly, if the consumer wants to remove an item from the buffer and sees that the buffer is empty, it goes to sleep until the producer puts something in the buffer and wakes it up.

This approach sounds simple enough, but it leads to race conditions. To keep track of the number of items in the buffer, a variable, count is needed.

- o If the maximum number of items the buffer can hold is N, the producer's code will first test to see if count is N.

- o If it is, the producer will go to sleep; if it is not, the producer will add an item and increment count.

The consumer's code is similar: first test count to see if it is 0.

- o If it is, go to sleep; if it is nonzero, remove an item and decrement the counter.

- o Each of the processes also tests to see if the other should be awakened, and if so, wakes it up.

The code for the producer and consumer process is shown in Figure 4.1

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 74

CSC312: Operating System II

Fig 4.1 The producer-consumer problem with a fatal race condition

4.1.1 Solving the Producer-Consumer Problem Using Semaphores

The producer and consumer processes share the following data structures:

```
int n;
semaphore mutex = 1;
semaphore empty = n;
semaphore full = 0
```

It is assumed that the pool consists of n buffers, each capable of holding one item. The mutex semaphore provides mutual exclusion for accesses to the buffer pool and is initialized to the value 1. The empty and full semaphores count the number of empty and full buffers. The semaphore empty is initialized to the value n; the semaphore full is initialized to the value 0.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 75

CSC312: Operating System II

- o The full and empty semaphores are needed to guarantee that certain event sequences do or do not occur.

o In this case, they ensure that the producer stops running when the buffer is full, and that the consumer stops running when it is empty.

The code for the producer process is shown in Figure 4.2, and the code for the consumer process is shown in Figure 4.3

```
do {
/* produce an item in next produced */
wait(empty);
wait(mutex);
/* add next produced to the buffer */
signal(mutex);
signal(full);
} while (true);
```

Fig 4.2 The structure of the producer process

```
do {
wait(full);
wait(mutex);
/* remove an item from buffer to next consumed */
signal(mutex);
signal(empty);
/* consume the item in next consumed */
} while (true);
```

Fig 4.3 The structure of the consumer process.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 76

CSC312: Operating System II

4.2 The Readers – Writers Problem

Suppose that a database is to be shared among several concurrent processes.

Some of these processes may want only to read the database, whereas others may want to update (that is, to read and write) the database. We distinguish between these two types of processes by referring to the former as readers and to the latter as writers.

Obviously, if two readers access the shared data simultaneously, no adverse effects will result. However, if a writer and some other process (either a reader or a writer) access the database simultaneously, chaos may ensue.

To ensure that these difficulties do not arise, it is required that the writers have exclusive access to the shared database while writing to the database. This synchronization problem is referred to as the readers – writers problem.

The readers – writers problem has several variations, all involving priorities.

The simplest one, referred to as the first readers – writers problem,

□ requires that no reader be kept waiting unless a writer has already obtained permission to use the shared object.

□ In other words, no reader should wait for other readers to finish simply because a writer is waiting.

The second readers – writers problem

□ requires that, once a writer is ready, that writer perform its write as soon as possible.

□ In other words, if a writer is waiting to access the object, no new readers may start reading.

A solution to either problem may result in starvation.

o In the first case, writers may starve;

o In the second case, readers may starve.

o For this reason, other variants of the problem have been proposed.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 77

CSC312: Operating System II

In the solution to the first readers – writers problem, the reader processes share the following data structures:

```
semaphore rw mutex = 1;
```

```
semaphore mutex = 1;
```

```
int read count = 0;
```

The semaphores mutex and rw mutex are initialized to 1; read count is initialized to 0.

□ The semaphore rw mutex is common to both reader and writer processes.

□ The mutex semaphore is used to ensure mutual exclusion when the variable read count is updated.

□ The read count variable keeps track of how many processes are currently reading the object.

□ The semaphore rw mutex functions as a mutual exclusion semaphore for the writers.

□ It is also used by the first or last reader that enters or exits the critical section.

□ It is not used by readers who enter or exit while other readers are in their critical sections.

The code for a writer process is shown in Figure 4.4

```
do {
    wait(rw mutex);
    /* writing is performed */
    signal(rw mutex);
} while (true);
```

Fig 4.4 The structure of a writer process.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 78

CSC312: Operating System II

The code for a reader process is shown in Figure 4.5.

```
do {
wait(mutex);
read count++;
if (read count == 1)
wait(rw mutex);
signal(mutex);
/* reading is performed */
wait(mutex);
read count--;
if (read count == 0)
signal(rw mutex);
signal(mutex);
} while (true);
```

Fig 4.5. The structure of a reader process

Note that:

- o if a writer is in the critical section and n readers are waiting, then one reader is queued on `rw mutex`, and $n - 1$ readers are queued on `mutex`.

- o when a writer executes `signal(rw mutex)`, we may resume the execution of either the waiting readers or a single waiting writer.

- The selection is made by the scheduler.

The readers - writers problem and its solutions have been generalized to provide reader - writer locks on some systems. Acquiring a reader - writer lock requires specifying the mode of the lock: either read or write access.

- When a process wishes only to read shared data, it requests the reader - writer lock in read mode.

- A process wishing to modify the shared data must request the lock in write mode.

- Multiple processes are permitted to concurrently acquire a reader - writer lock in read mode, but only one process may

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 79

CSC312: Operating System II

acquire the lock for writing, as exclusive access is required for writers.

Reader - writer locks are most useful in the following situations:

- In applications where it is easy to identify which processes only read shared data and which processes only write shared data.

- In applications that have more readers than writers.

4.3. The Dining-Philosophers Problem

The dining-philosophers problem is considered a classic synchronization

problem that presents a simple representation of the need to allocate several resources among several processes in a deadlock-free and starvation-free manner. The problem can be stated quite simply as follows. Five philosophers are seated around a circular table. Each philosopher has a plate of spaghetti. The spaghetti is so slippery that a philosopher needs two forks to eat it. Between each pair of plates is one fork.

The life of a philosopher consists of alternating periods of eating and thinking. When a philosopher gets sufficiently hungry, she tries to acquire her left and right forks, one at a time, in either order. If successful in acquiring two forks, she eats for a while, then puts down the forks, and continues to think.

The layout of the table is illustrated in Figure 4.6 using rice for spaghetti and chopsticks for forks.

Fig 4.6 Lunch time in the Philosophy Department

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 80

CSC312: Operating System II

One simple solution is to represent each chopstick with a semaphore. A philosopher tries to grab a chopstick by executing a wait() operation on that semaphore. She releases her chopsticks by executing the signal() operation on the appropriate semaphores.

Thus, the shared data are

semaphore chopstick[5];

where all the elements of chopstick are initialized to 1

The structure of philosopher i is shown in Figure 4.7

```
do {
wait(chopstick[i]);
wait(chopstick[(i+1) % 5]);
/* eat for awhile */
signal(chopstick[i]);
signal(chopstick[(i+1) % 5]);
/* think for awhile */
} while (true);
```

Figure 4.7 The structure of philosopher i.

Although this solution guarantees that no two neighbors are eating simultaneously, it nevertheless must be rejected because it could create a deadlock.

□ Suppose that all five philosophers become hungry at the same time and each grabs her left chopstick. All the elements of chopstick will now be equal to 0.

□ When each philosopher tries to grab her right chopstick, she will be delayed forever.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 81

CSC312: Operating System II

4.3.1 Dining-Philosophers Solution Using Monitors

A monitor is a collection of procedures, variables, and data structures that are all grouped together in a special kind of module or package.

o Processes may call the procedures in a monitor whenever they want to, but they cannot directly access the monitor's internal data structures from procedures declared outside the monitor

The monitor solution imposes the restriction that a philosopher may pick up her chopsticks only if both of them are available. To code this solution, we need to distinguish among three states in which we may find a philosopher.

For this purpose, we introduce the following data structure:

```
enum {THINKING, HUNGRY, EATING} state[5];
```

Philosopher i can set the variable $state[i] = EATING$ only if her two neighbors are not eating: $(state[(i+4) \% 5] \neq EATING)$ and $(state[(i+1) \% 5] \neq EATING)$.

Figure 4.8 shows a monitor solution to the dining-philosopher problem
monitor DiningPhilosophers

```
enum {THINKING, HUNGRY, EATING} state[5];
condition self[5];

void pickup(int i) {
    state[i] = HUNGRY;
    test(i);
    if (state[i] != EATING)
        self[i].wait();

    void putdown(int i) {
        state[i] = THINKING;
        test((i + 4) % 5);
        test((i + 1) % 5);

        void test(int i) {
            if ((state[(i + 4) % 5] != EATING) && (state[i] ==
                HUNGRY) &&
                (state[(i + 1) % 5] != EATING)) {
                state[i] = EATING;
                self[i].signal();
```

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 82

CSC312: Operating System II

```
initialization code() {
    for (int i = 0; i < 5; i++)
        state[i] = THINKING;
```

Fig 4.8 A monitor solution to the dining-philosopher problem

The condition `self[5]` allows philosopher `i` to delay herself when she is hungry but is unable to obtain the chopsticks she needs.

The distribution of the chopsticks is controlled by the monitor

DiningPhilosophers, whose definition is shown in Figure 4.8.

□ Each philosopher, before starting to eat, must invoke the operation `pickup()`.

□ This act may result in the suspension of the philosopher process.

□ After the successful completion of the operation, the philosopher may eat.

□ Following this, the philosopher invokes the `putdown()` operation.

□ Thus, philosopher `i` must invoke the operations `pickup()` and `putdown()` in the following sequence:

```
DiningPhilosophers.pickup(i);
```

```
eat
```

```
DiningPhilosophers.putdown(i);
```

It is easy to show that this solution ensures that no two neighbors are eating simultaneously and that no deadlocks will occur. However, it is possible for a philosopher to starve to death. The monitor mechanism can be implemented using semaphores.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 83

CSC312: Operating System II

Exercises

The readers and writers problem can be formulated in several ways with regard to which category of processes can be started when. Carefully describe three different variations of the problem, each one favoring (or not favoring) some category of processes. For each variation, specify what happens when a reader or a writer becomes ready to access the database, and what happens when a process is finished

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 84

Paper 3 of 3 — CSC 312 MATERIAL

Level 300 • First Semester • Handout

CSC312: Operating System II

Department of Computer Science

Federal University of Petroleum Resources, Effurun,

Delta State

CSC312:

OPERATING SYSTEM II

[ORHIONKPAIYO B. C]

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 1

CSC312: Operating System II

1. COURSE GOAL

The goal of this course is to extend the students' fundamental knowledge in operating systems. It intends to acquaint students with the mechanisms involved in process and memory management in contemporary Operating Systems.

2. COURSE OBJECTIVES

At the end of this course, students should be able to:

- i. Describe the various ways of organizing memory hardware.
- ii. Explain various techniques of allocating memory to processes.
- iii. Discuss in detail how paging works in contemporary computer systems.
- iv. Discuss both software and hardware solutions of the critical-section problem.
- v. Examine some classical process-synchronization problems.
- vi. Examine the tools that are used to solve process synchronization problems
- vii. Describe various CPU-scheduling algorithms.
- viii. Discuss evaluation criteria for selecting a CPU-scheduling algorithm for a particular system.
- ix. Examine the scheduling algorithms of some operating systems.
- x. Discuss the placement algorithms of some memory management techniques
- xi. Explain the causes and types of fragmentation and their possible solutions
- xii. Differentiate between logical address and physical address
- xiii. Explain the term race condition and state its causes
- xiv. State the critical section problem and illustrate the different methods for solving it
- xv. Enumerate the conditions that must hold to achieve a good solution to critical section problem
- xvi. Different between semaphore and mutex locks and illustrate how each solves the critical section problem
- xvii. State the adverse effect of busy waiting in multiprogramming systems
- xviii. Explain the cause of deadlock and illustrate the concept using two processes
- xix. List the classical problems that are commonly used to illustrate the power of

synchronization primitives

xx. Illustrate classical synchronization problems using appropriate synchronization primitives

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 2

CSC312: Operating System II

3. COURSE DESCRIPTION

Concurrency: States & State diagrams Structures, Dispatching and Context Switching; interrupts; Concurrent execution; Mutual exclusion problem and some solutions Deadlock; Models and mechanisms (Semaphores, monitors etc.)

Producer – Consumer Problems & Synchronization. Multiprocessor issues. Scheduling & Despatching. Memory Management: Overlays, Swapping and Partitions, Paging & Segmentations Placement & replacement policies, working sets and Trashing, Caching.

4. COURSE SCHEDULE

WEEK TOPICS SUB TOPICS

1 Course Orientation and the syllabus Review of course syllabus

2– 4 Memory Management i. Memory management requirement

ii. Contiguous memory allocation

iii. Multiple partitioning

iv. Swapping

v. Paging

vi. Segmentation

vii. Virtual Memory

5 –7 CPU Scheduling i. Types of CPU scheduling

ii. Criteria for Selecting a CPU-

scheduling algorithm

iii. Scheduling algorithms

a. First-Come,First-Served

Scheduling

b. Shortest-Job-First Scheduling

c. Priority Scheduling

d. Round-Robin Scheduling

e. Multi-level Queue Scheduling

f. Multilevel feedback Queue

Scheduling

8 - 10 Mutual Exclusion & Synchronization i. Principles of concurrency

ii. Mutual exclusion hardware support

iii. Semaphores

iv. Monitors

v. Message passing

vi. Reader/Writer problem

- 11 - 13 Deadlock & Starvation i. Principles of deadlock
- ii. Deadlock prevention
- iii. Deadlock Avoidance
- iv. Dining Philosophers problem

14 Test & Revision

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 3

CSC312: Operating System II

5. CONTACTS

6. GRADING

Attendance - 3%

Assignments/Laboratory - 15%

Test - 12%

Exam - 70%

7. POLICIES

(i) Attendance - It is expected that every student will be in class for lectures. Attendance records will be kept and used to award marks for each student. Only students with up to or above 75% attendance will be awarded 3mks. Signing for friends is an act of DISHONESTY and it is HIGHLY PROHIBITED.

(ii) Makeup test - There shall be no makeup test. The date for the test shall be announced to the class at least one week before the test.

(iii) Academic Integrity - Any form of academic dishonesty, such as copying during test /exam or making copies of other students' work, is prohibited.

(iv). Submission of Assignments/Lab work - Late submission of assignment or lab work will be graded as Score - x, where x is the number of days of late submission.

9. TEXTBOOKS

1. Operating System Concepts, Abraham Silberchatz, Peter B. Galvin, Greg Gagne, Ninth Edition, 2012, Wiley.

2. Operating Systems: Internals & Design Principles - Williams Stallings

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 4

CSC312: Operating System II

1. MEMORY MANAGEMENT

The main memory is central to the operation of a modern computer system. The CPU loads instructions only from memory, so any programs to run must be stored there. Main memory is a repository of quickly accessible data shared by the CPU and I/O devices.

The operating system is responsible for the following activities in connection with memory management:

- Keeping track of which parts of memory are currently being used and who is using them
- Deciding which processes (or parts of processes) and data to move into and out of memory
- Allocating and deallocating memory space as needed

Thus, memory management is the task carried out by the Operating System and hardware to accommodate multiple processes in main memory. If only a few processes can be kept in main memory, then much of the time all processes will be waiting for I/O and the CPU will be idle.

o The purpose of memory management is to ensure fair, secure, orderly, and efficient use of memory.

To improve both the utilization of the CPU and the speed of the computer's response to its users,

o general-purpose computers must keep several programs in memory, creating a need for memory management.

o thus, memory needs to be allocated efficiently in order to have as many processes in memory as possible

o Many different memory management schemes are used. These schemes reflect various approaches, and the effectiveness of any given algorithm depends on the situation.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 5

CSC312: Operating System II

□ In most schemes, the kernel occupies some fixed portion of main memory and the rest is shared by multiple processes

General-purpose computers run most of their programs from rewritable memory, called main memory (also called Random-Access Memory, or RAM). The programs and data do not reside in main memory permanently. This arrangement usually is not possible for the following two reasons:

1. Main memory is usually too small to store all needed programs and data permanently.
2. Main memory is a volatile storage device that loses its contents when power is turned off or otherwise lost.

Thus, most computer systems provide secondary storage as an extension of main memory, thus providing virtual memory system

o Virtual memory allows the computer to use part of a permanent storage device (such as a hard disk) as extra memory.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 6

CSC312: Operating System II

1.1 Memory Management Requirements

It is helpful to keep in mind the requirements that memory management intended to satisfy. These requirements include the following:

- i. Relocation
- ii. Protection
- iii. Sharing
- iv. Logical organization
- v. Physical organization

□ Relocation

The memory available has to be shared among various processes present in a multiprogramming system. Thus, it is impossible to determine what other programs would reside in the main memory during the program execution.

If the active processes are swapped in and out of the main memory, it lets the OS have a bigger and better pool of processes that are ready to execute.

Whenever any program happens to swap out to its disk memory, it would not always be possible that swapping it back into the main memory would let it occupy the previous memory location. It is because the location might still be occupied by some other process. The given processes might have to relocate to a different section of the memory.

After the program gets loaded into the main memory, the OS and the processor should be able to translate the logical addresses into the physical addresses

□ The address generated by the CPU is said to be a logical address. An address generated by MMU is called a physical address

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 7

CSC312: Operating System II

□ Protection

Every process has to be protected against all unwanted interferences when all the other processes try to write into a process, whether it is accidental or incidental.

Processes should not be able to reference memory locations in another process without permission. It must be checked at run time.

The memory protection requirement must be satisfied by the processor (hardware) rather than the operating system (software).

The operating can protect the memory with the help of base and

limit register.

The requirement of memory protection has to be satisfied by the processor instead of the OS, since the OS can hardly control any given process whenever it occupies the processor.

□ Sharing

Some protection mechanisms allow various processes to access a similar section of main memory. Thus, it allows all the processes to access the very same copy of the program instead of having their own separate copy, which has an advantage.

For instance, numerous processes may utilize the very same system file. It is natural to load a single copy of the given file in the main memory and to let it be shared by these processes.

- o It is the memory management's task to allow the shared areas of memory with controlled access without compromising the protection.

Various mechanisms are utilized here to support the relocation-supported sharing abilities.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 8

CSC312: Operating System II

□ Logical Organization

The main memory is organized as a linear form, or it could be an address space that's one-dimensional. It comprises a sequence of words or bytes. Programs are written in modules. Modules can be written and compiled independently.

- o Different degrees of protection are given to modules (read-only, execute-only). Modules are shared among processes.

□ Physical Organization

A computer memory's structure has two levels, and they are referred to as the main memory and the secondary memory. The secondary memory is mainly used for data storage on a long-term basis, but the main memory holds the programs currently in use.

- o moving information between these two levels of memory is a major concern of memory management (OS)

- it is highly inefficient to leave this responsibility to the application programmer

- Memory must be fairly allocated for high processor utilization for systematic flow of information between main and secondary memory

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 9

CSC312: Operating System II

1.2 Basics of memory Management

In order to understand the concept of memory management, we will discuss some issues that are pertinent to managing memory:

- basic hardware,
- binding symbolic memory addresses to actual physical addresses,
- the distinction between logical and physical addresses, and
- dynamic linking and shared libraries.

1.2.1 Basic Hardware

Memory & Registers:

Main memory and the registers built into the processor itself are the only general-purpose storage that the CPU can access directly.

All forms of memory provide an array of bytes. Each byte has its own address.

Interaction is achieved through a sequence of load or store instructions to specific memory addresses.

- o The load instruction moves a byte or word from main memory to an internal register within the CPU, whereas
- o The store instruction moves the content of a register to main memory

There are machine instructions that take memory addresses as arguments, but none that take disk addresses.

- o Therefore, any instructions in execution, and any data being used by the instructions, must be in one of these direct-access storage devices.

If the data are not in memory, they must be moved there before the CPU can operate on them.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 10

CSC312: Operating System II

Cache:

Cache sits between main memory and CPU registers. Registers that are built into the CPU are generally accessible within one cycle of the CPU clock. Completing a memory access may take many cycles of the CPU clock.

- o In such cases, the processor normally needs to stall, since it does not have the data required to complete the instruction that it is executing.

□ This situation is intolerable because of the frequency of memory accesses.

- o The remedy is to add fast memory (or cache) between the CPU and main memory, typically on the CPU chip for fast access.

Base and Limit Registers: A pair of base and limit registers define the logical address space.

- o The base register holds the smallest legal physical memory address;
- o The limit register specifies the size of the range.

For example, if the base register holds 300040 and the limit register is 120900, then

the program can legally access all addresses from 300040 through 420939 (inclusive).

CPU must check every memory access generated in user mode to be sure it is between base and limit for that user, thus protecting the memory space

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 11

CSC312: Operating System II

- o Any attempt by a program executing in user mode to access operating-system memory or other users' memory results in a trap to the operating system, which treats the attempt as a fatal error (see diagram below)

The base and limit registers can be loaded only by the operating system, which uses a special privileged instruction.

- o Since privileged instructions can be executed only in kernel mode, and since only the operating system executes in kernel mode, only the operating system can load the base and limit registers.

- o This scheme prevents a user program from (accidentally or deliberately) modifying the code or data structures of either the operating system or other users.

1.2.2 Address Binding

Usually, a program resides on a disk as a binary executable file. To be executed, the program must be brought into memory and placed within a process.

- o The processes on the disk that are waiting to be brought into memory for execution form the input queue or job queue.

The normal single-tasking procedure is to select one of the processes in the input queue and to load that process into memory.

- o As the process is executed, it accesses instructions and data from memory.
- o Eventually, the process terminates, and its memory space is declared available.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 12

CSC312: Operating System II

Addresses in the source program are generally symbolic (such as the variable count).

- o A compiler typically binds these symbolic addresses to relocatable addresses

- o The linkage editor or loader in turn binds the relocatable addresses to absolute addresses.

Each binding is a mapping from one address space to another.

Classically, the binding of instructions and data to memory addresses can be done at any step along the way:

- Compile time: If memory location known a priori, absolute code can be

generated; must recompile code if starting location changes

- o For example, if you know that a user process will reside starting at location R, then the generated compiler code will start at that location and extend up from there. If, at some later time, the starting location changes, then it will be necessary to recompile this code.

- Load time: Must generate relocatable code if memory location is not known at compile time

- Execution time: Binding delayed until run time if the process can be moved during its execution from one memory segment to another. Need hardware support for address maps (e.g., base and limit registers).

1.2.3 Logical Versus Physical Address Space

An address generated by the CPU is commonly referred to as a logical address also referred to as virtual address, whereas an address seen by the memory unit—that is, the one loaded into the memory-address register of the memory—is commonly referred to as a physical address.

- o Logical and physical addresses are the same in compile-time and load-time address-binding schemes; logical (virtual) and physical addresses differ in execution-time address-binding scheme

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 13

CSC312: Operating System II

- o The set of all logical addresses generated by a program is a logical address space.

- o The set of all physical addresses corresponding to these logical addresses is a physical address space.

Program issues addresses in a logical/virtual address space which must be translated to physical address space

- o Think of the program as having a contiguous logical/virtual address space that starts at 0,

- o and a contiguous physical address space that starts somewhere else

Thus, in the execution-time address-binding scheme, the logical and physical address spaces differ.

- o The run-time mapping from virtual to physical addresses is done by a hardware device in the CPU called the memory-management unit (MMU).

The MMU has two special registers that are accessed by the CPU's control unit.

- o A data to be sent to main memory or retrieved from memory is stored in the Memory Data Register (MDR).

- o The desired logical memory address is stored in the Memory Address Register (MAR).

In MMU scheme, the value in the relocation register (i.e. the base register) is added to every address generated by a user process at the time it is sent to memory.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 14

CSC312: Operating System II

□ For example, if the base is at 14000, then an attempt by the user to address location 0 is dynamically relocated to location 14000; an access to location 346 is mapped to location 14346.

□ We now have two different types of addresses:

- o logical addresses (in the range 0 to max) and
- o physical addresses (in the range $R + 0$ to $R + \text{max}$ for a base value R).

1.2.4 Dynamic Loading

With dynamic loading, a routine is not loaded until it is called.

- o All routines are kept on disk in a re-locatable load format.
- o The main program is loaded into memory and is executed.

When a routine needs to call another routine, the calling routine first checks to see whether the other routine has been loaded or not

o If it has not, the re-locatable linking loader is called to load the desired routine into memory and to update the program's address tables to reflect this change. Then control is passed to the newly loaded routine.

Dynamic loading does not require special support from the operating system.

- o It is the responsibility of the users to design their programs to take advantage of such a method.
- o Operating systems may help the programmer, however, by providing library routines to implement dynamic loading.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 15

CSC312: Operating System II

The advantage of dynamic loading is that a routine is loaded only when it is needed. This method is particularly useful when large amounts of code are needed to handle infrequently occurring cases, such as error routines.

o In this case, although the total program size may be large, the portion that is used (and hence loaded) may be much smaller.

1.2.5 Dynamic Linking and Shared Libraries

Dynamically linked libraries are system libraries that are linked to user programs when the programs are run.

- o A dynamic link library (DLL) is a collection of small programs that can be loaded when needed by larger programs and used at the same time.
- o The small program lets the larger program communicate with a specific device, such as a printer or scanner.
- o It is often packaged as a DLL program, which is usually referred to as a DLL file.

One of the main tasks for linker is to make code of library functions (eg printf(), scanf(), sqrt(), ..etc) available to user program.

Some operating systems support only static linking, in which system libraries are treated like any other object module and are combined by the loader into the binary program image (e.g. .lib files in Windows)

Dynamic linking, in contrast, is similar to dynamic loading. Here, though, linking, rather than loading, is postponed until execution time.

- o This feature is usually used with system libraries, such as language subroutine libraries.

- o With dynamic linking, a stub is included in the image for each library routine reference.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 16

CSC312: Operating System II

□ The stub is a small piece of code that indicates how to locate the appropriate memory-resident library routine or how to load the library if the routine is not already present.

Unlike dynamic loading, dynamic linking and shared libraries generally require help from the operating system.

- o If the processes in memory are protected from one another, then the operating system is the only entity that can check to see whether the needed routine is in another process's memory space or that can allow multiple processes to access the same memory addresses.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 17

CSC312: Operating System II

1.3 Methods of Memory Management

Memory management methods include:

- i. Swapping
- ii. Partitioning
- iii. Segmentation
- iv. Paging, and
- v. Virtual memory

1.3.1 Swapping

A process needs to be in the memory to be executed. What if there is no enough memory for all processes? A process, however, can be swapped temporarily out of memory to a backing store, and then brought back into memory for continued execution.

- o Backing store is a fast disk large enough to accommodate copies of all memory images for all users; it must provide direct access to these memory images.

- o Swapping makes it possible for the total physical address space of all processes to exceed the real physical memory of the system, thus increasing

the degree of multiprogramming in a system.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 18

CSC312: Operating System II

The system maintains a ready queue of all processes whose memory images are on the backing store or in memory and are ready to run.

Whenever the CPU scheduler decides to execute a process, it calls the dispatcher.

- o The dispatcher checks to see whether the next process in the queue is in memory.

- o If it is not, and if there is no free memory region, the dispatcher swaps out a process currently in memory and swaps in the desired process.

- o It then reloads registers and transfers control to the selected process.

The context-switch time in such a swapping system is fairly high.

- o To get an idea of the context-switch time, let's assume that the user process is 100 MB in size and the backing store is a standard hard disk with a transfer rate of 50 MB per second.

- o The actual transfer of the 100-MB process to or from main memory takes $100 \text{ MB} / 50 \text{ MB per second} = 2 \text{ seconds}$

The swap time is 2000 milliseconds (i.e. 2000ms)

- o Since we must swap both out and in, the total swap time is about 4,000 milliseconds.

The total transfer time is directly proportional to the amount of memory swapped.

- o If we have a computer system with 4 GB of main memory and a resident operating system taking 1 GB, the maximum size of the user process is 3 GB.

To reduce the cost of memory swap time,

- o by knowing how much memory is really being used

- o System calls to inform OS of memory use via `request_memory()` and `release_memory()`

Standard swapping is not used in modern operating systems, but modified version common

- Swap only when free memory extremely low

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 19

CSC312: Operating System II

Swapping on Mobile Systems

Although most operating systems for PCs and servers support some modified version of swapping,

- o Mobile systems typically do not support swapping in any form.

- Mobile devices generally use flash memory rather than more spacious hard disks as their persistent storage.

- o The resulting space constraint is one reason why mobile operating-system

designers avoid swapping.

o Other reasons include

- the limited number of writes that flash memory can tolerate before it becomes unreliable and
- the poor throughput between main memory and flash memory in these devices.

Instead other methods are used to free memory if low

o iOS asks apps to voluntarily relinquish allocated memory

- Read-only data thrown out and reloaded from flash if needed
- Failure to free can result in termination

o Android terminates apps if low free memory, but first writes application state to flash for fast restart

1.3.2 Partitioning

Single Partitioning (Contiguous Allocation)

The main memory must accommodate both operating system and the various user spaces.

The memory is usually divided into two partitions:

- one for the resident operating system and
- one for the user processes.

The operating system may be placed in the high memory or the low memory.

o The position of the interrupt vector usually affects this decision.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 20

CSC312: Operating System II

Since the interrupt vector is often in the low memory, programmers place the OS in low memory too.

o It is desirable to have several user processes residing in the memory at the same time.

In contiguous memory allocation, each process is contained in a single contiguous section of memory.

o Relocation registers are used to protect user processes from each other, and from changing operating-system code and data

- Base register contains value of smallest physical address (for example, relocation = 100040).
- Limit register contains range of logical addresses – each logical address must be less than the limit register (for example, limit = 74600).

The MMU maps the logical address dynamically by adding the value in the relocation register. This mapped address is sent to memory as shown below

Hardware support for relocation and limit registers

Multiple Partitioning

In a uni-programming system, main memory is divided into two parts: one part for the operating system (resident monitor, kernel) and one part for the program currently being executed. In a multi-programming system, the “user” part of memory must be further subdivided to accommodate multiple processes.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 21

CSC312: Operating System II

There are variations of achieving this multiple partitioning. These include:

- i. Fixed partitioning
- ii. Variable partitioning

□ Fixed Partitioning

Physical memory is broken up into fixed partitions

- o partitions may have different sizes, but partitioning never changes
- o Each partition may contain exactly one process. Thus, the degree of multiprogramming is bound by the number of partitions.
- o When a partition is free, a process is selected from the input queue and is loaded into the free partition. When the process terminates, the partition becomes available for another process.
- o This method was originally
 - used by the IBM OS/360 operating system (called MFT) but is no longer in use.

There are variations of fixed partitioning:

- i. Equal-size partitions
 - o Any process whose size is less than or equal to the partition size can be loaded into an available partition
 - o The operating system can swap out a process if all partitions are full and no process is in the Ready or Running state

Disadvantages of equal-size partitioning

- A program may be too big to fit in a partition
- Main memory utilization is inefficient.
- o Any program, regardless of size, occupies an entire partition
- o Internal fragmentation. Waste space due to the block of data loaded being smaller than the partition

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 22

CSC312: Operating System II

- ii. Unequal-size partitioning

Assign each process the smallest partition to which it will fit

Advantages:

- Process are always assigned in such a way as to minimize wasted memory within a partition → internal fragmentation

□ Relatively simple and require minimal OS software and overhead

Disadvantages:

□ Limitations on the active number of processes, number of partitions specified at system generation time

□ Small jobs cannot utilize partition space efficiently; In most cases it is an inefficient technique

Memory assignment for fixed partitioning

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 23

CSC312: Operating System II

Placement Algorithm for fixed partition

◆ Equal-size partitions

o Because all partitions are of equal size, it does not matter which partition is used

◆ Unequal-size partitions

o Can assign each process to the smallest partition within which it will fit

o Queue for each partition size

o Processes are assigned in such a way as to minimize wasted memory within a partition

Variable/Dynamic Partitioning

Physical memory is broken up into partitions dynamically – partitions are tailored to programs. Partitions are of variable length and number

o Process is allocated exactly as much memory as it requires

o When a process arrives, it is allocated memory from a hole large enough to accommodate it

□ A Hole is a block of available memory; holes of various sizes are scattered throughout memory

o The operating system keeps a table indicating which parts of memory are available and which are occupied.

o This technique was used by IBM's mainframe operating system, OS/MVT

Disadvantages of Dynamic partitioning

o Memory becomes more and more fragmented with time

o Memory utilization declines

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 24

CSC312: Operating System II

Effect of dynamic partitioning

Placement Algorithms for Dynamic Partitioning

First fit: Allocate the first hole that is big enough.

o Searching can start either at the beginning of the set of holes or at the location where the previous first-fit search ended.

- o We can stop searching as soon as we find a free hole that is large enough.
- o The idea is to minimize the time to analyze the memory space available

Best fit: Allocate the smallest hole that is big enough.

- o One must search the entire list, unless the list is ordered by size.
- o This strategy produces the smallest leftover hole.
- o The Idea is to minimize wastage of free memory space

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 25

CSC312: Operating System II

Worst fit: Allocate the largest hole.

- o Again, one must search the entire list, unless it is sorted by size.
- o This strategy produces the largest leftover hole, which may be more useful than the smaller leftover hole from a best-fit approach.
- o The idea is to increase the possibility that another process can use the left-over space

The example below illustrate the allocation of 12kb to a process using the three algorithms

Fragmentation

The allocation and de-allocation of memory creates a condition called fragmentation. This means that there are lots of small fragments (or holes) of free memory that cannot be used by any other process. This results in a reduction in the amount of total available memory.

- External Fragmentation – Gaps between allocated contiguous memory - total memory space exists to satisfy a request, but it is not contiguous
- o Dynamic partitioning problem

- Internal Fragmentation – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used
- o Fixed partitioning problem

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 26

CSC312: Operating System II

How to solve fragmentation problem

Reduce external fragmentation by compaction

- Shuffle memory contents to place all free memory together in one large block
- o Swap a program out
- o Re-load it, adjacent to another
- o Adjust its base register
- Compaction is possible only if relocation is dynamic, and is done at execution time

Exercise

Given free memory partitions of 100K, 500K, 200K, 300K, and 600K (in order),

- i. How would each of the First-fit, Best-fit, and Worst-fit algorithms place processes of 212K, 417K, 112K, and 426K (in order)?
- ii. Which algorithm makes the most efficient use of memory?

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 27

CSC312: Operating System II

1.3.3 Paging

Paging is a technique of memory management in which physical memory is broken into blocks of the same size called pages.

The basic method for implementing paging involves breaking

- o physical memory into fixed-sized blocks called frames and
- o logical memory into blocks of the same size called pages.

When a process is to be executed, its corresponding pages are loaded into any available memory frames.

- A frame is a fixed-length block of main memory
- o Operating system keeps track of all free frames.
- o Operating system needs n free frames to run a program of size n pages.

The hardware support for paging is illustrated in the diagram below. Every address generated by the CPU is divided into two parts:

- o Page number (p) -- page number is used as an index into a page table which contains base address of each page in physical memory.
- o Page offset (d) -- page offset is combined with base address to define the physical memory address.

Paging Hardware

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 28

CSC312: Operating System II

The paging model of memory is shown in the following diagram

Paging model of logical and physical memory

The page size (like the frame size) is defined by the hardware. The size of a page is a power of 2, varying between 512 bytes and 1 GB per page, depending on the computer architecture.

- o The selection of a power of 2 as a page size makes the translation of a logical address into a page number and page offset particularly easy.
- o If the size of the logical address space is 2^m , and a page size is 2^n bytes, then the high-order $m - n$ bits of a logical address designate the page number, and the n low-order bits designate the page offset.

Thus, the logical address is as follows:

where p is an index into the page table and d is the displacement within the page.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 29

CSC312: Operating System II

As a concrete example, consider the memory in the diagram below. Here, in the logical address, $n = 2$ and $m = 4$. Using a page size of 4 bytes and a physical memory of 32 bytes (8 pages), we show how the programmer's view of memory can be mapped into physical memory.

Logical address 0 is page 0, offset 0.

- o Indexing into the page table, we find that page 0 is in frame 5.

□ Thus, logical address 0 maps to physical address 20 [= (5 × 4) + 0].

- o Logical address 3 (page 0, offset 3) maps to physical address 23 [= (5 × 4) + 3].

- o Logical address 4 is page 1, offset 0; according to the page table, page 1 is mapped to frame 6.

□ Thus, logical address 4 maps to physical address 24 [= (6 × 4) + 0].

- o Logical address 13 maps to physical address 9.

Paging example for a 32-byte memory with 4-byte pages

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 30

CSC312: Operating System II

Advantages of Paging

- o Easy to allocate: keep a free list of available pages and grab the first one.
- o Easy to swap since everything is the same size, which is usually the same size as disk blocks to and from which pages are swapped.
- o External fragmentation is avoided by using paging technique
- any free frame can be allocated to a process that needs it.

Disadvantages of Paging

- o Efficiency of access: even small page tables are generally too large to load into fast memory in the relocation box. Instead, page tables are kept in main memory and the relocation box only has the page table's base address. It thus takes one overhead reference for every real memory reference.
- o Internal fragmentation: page size doesn't match up with information size. The larger the page, the worse this is. But no external fragmentation.

1.3.4 Segmentation

Segmentation supports the user-view of memory that the logical address space becomes a collection of (typically disjoint) segments.

- o A segment is a variable-length block of data that resides in secondary memory
- o Segments have a name (or a number) and a length.
- o Addresses specify segment, and offset within segment

- o To access memory, user program specifies segment + offset

- Logical address = segment name (number) + offset

Segmentation permits the physical address space of a process to be non-contiguous.

Typical segments include

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 31

CSC312: Operating System II

- o global variables

- o procedure call stack

- o code for each function

- o local variables for each large data structures

Segmentation hardware

In the segmentation hardware above, each entry in the segment table has a segment base and a segment limit.

- o The segment base contains the starting physical address where the segment resides in memory, and the segment limit specifies the length of the segment.

- o A logical address consists of two parts: a segment number, and an offset into that segment

- The segment number is used as an index to the segment table.

- The offset of the logical address must be between 0 and the segment limit. If it is not, we trap to the operating system (logical addressing attempt beyond end of segment).

- o When an offset is legal, it is added to the segment base to produce the address in physical memory of the desired byte.

- o The segment table is thus essentially an array of base-limit register pairs

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 32

CSC312: Operating System II

Example to illustrate Segmentation:

We have five segments numbered from 0 through 4. The segments are stored in physical memory as shown in the diagram below.

The segment table has a separate entry for each segment, giving the beginning address of the segment in physical memory (or base) and the length of that segment (or limit).

For example,

- segment 2 is 400 bytes long and begins at location 4300. Thus, a reference to byte 53 of segment 2 is mapped onto location $4300 + 53 = 4353$.

- A reference to segment 3, byte 852, is mapped to 3200 (the base of segment 3) + 852 = 4052.

- A reference to byte 1222 of segment 0 would result in a trap to the operating system, as this segment is only 1,000 bytes long.

Example of segmentation

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 33

CSC312: Operating System II

Advantages:

- o Segments can be shared, swapped, and assigned to storage independently.
- o Segment has defined length so it is possible to detect access outside of the segment.

□ It throws new type of error – segmentation fault

Disadvantages

- o It is not easy to place the segment into memory
- Segments has different size
- Memory fragmentation
- Segment moving has big overhead (one comparison, one addition)

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 34

CSC312: Operating System II

Exercises

1. Consider the following segment table:

Segment Base Length

0 219 600

1 2300 14

2 90 100

3 1327 580

4 1952 96

What are the physical addresses for the following logical addresses?

- a. 0,430
- b. 1,10
- c. 2,500
- d. 3,400
- e. 4,112

2. Assuming a 1-KB page size, what are the page numbers and offsets for the following address references (provided as decimal numbers):

- a. 3085
- b. 42095
- c. 215201
- d. 650000
- e. 2000001

3. Although Android does not support swapping on its boot disk, it is possible to set up a swap space using a separate SD non-volatile memory card. Why would Android disallow swapping on its boot disk yet allow it on a secondary disk?

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 35

CSC312: Operating System II

1.3.5 Virtual Memory

Despite the fact that most modern computers have between 1 GB and 4 GB of RAM, it is possible to run out of memory. This can happen when too many processes are running at the same time, or when a process requires more memory than is currently available (unused). One solution to this problem is to use virtual memory.

- o Virtual memory is a technique that allows the execution of processes that are not completely in memory

This means that the operating system's memory manager will use a portion of another storage device to act as if it is extra RAM.

- o In most cases, this virtual memory space will be on a permanent storage device such as a hard disk.

- However, some newer operating systems (such as Windows Vista and Windows 7) also allow one to use a removable media device like a USB flash drive as virtual memory (called Windows Ready Boost).

- o The memory manager will manage this resource so that it gives the illusion that the computer has more RAM than is actually installed.

Advantages

- o One major advantage of this scheme is that programs can be larger than physical memory.
- o Further, virtual memory abstracts main memory into an extremely large, uniform array of storage, separating logical memory as viewed by the user from physical memory.
- o This technique frees programmers from the concerns of memory-storage limitations.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 36

CSC312: Operating System II

Disadvantages

- o Virtual memory is not easy to implement
- o May substantially decrease performance if it is used carelessly.

Implementation of Virtual Memory

One common technique for implementing virtual memory is demand paging.

- o With demand-paged virtual memory, pages are loaded only when they are demanded during program execution.
- o Pages that are never accessed are thus never loaded into physical memory.

A demand-paging system is similar to a paging system with swapping (see diagram below) where processes reside in secondary memory (usually a disk). When a

process is to be executed, it is swapped into memory. Rather than swapping the entire process into memory, though, a pager (popularly called lazy swapper) is used.

- o A pager is concerned with swapping the individual pages of a process that will be needed.

- o It never swaps a page into memory unless that page will be needed.

Transfer of a paged memory to contiguous disk space.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 37

CSC312: Operating System II

When a process is to be swapped in, the pager guesses which pages will be used before the process is swapped out again

- o Instead of swapping in a whole process, the pager brings only those pages into memory.

- o Thus, it avoids reading into memory pages that will not be used anyway, decreasing the swap time and the amount of physical memory needed.

With this scheme, some forms of hardware support are needed to distinguish between the pages that are in memory and the pages that are on the disk.

The valid-invalid bit scheme is used for this purpose.

- o When this bit is set to “valid,” the associated page is both legal and in memory.

- o If the bit is set to “invalid,” the page either is not valid (that is, not in the logical address space of the process) or is valid but is currently on the disk.

The page-table entry for a page that is brought into memory is set as usual, but the page-table entry for a page that is not currently in memory is either simply marked invalid or contains the address of the page on disk. This situation is depicted in the diagram below

Page table when some pages are not in main memory

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 38

CSC312: Operating System II

But what happens if the process tries to access a page that was not brought into memory?

- o Access to a page marked invalid causes a page fault.

The paging hardware, in translating the address through the page table, will notice that the invalid bit is set, causing a trap to the operating system.

- o This trap is the result of the operating system’s failure to bring the desired page into memory.

The procedure for handling this page fault is straightforward (See diagram below):

- We check an internal table (usually kept with the process control block) for this process to determine whether the reference was a valid or an invalid memory access.

- If the reference was invalid, we terminate the process. If it was valid but we have not yet brought in that page, we now page it in.
- We find a free frame (by taking one from the free-frame list, for example).
- We schedule a disk operation to read the desired page into the newly allocated frame.
- When the disk read is complete, we modify the internal table kept with the process and the page table to indicate that the page is now in memory.
- We restart the instruction that was interrupted by the trap. The process can now access the page as though it had always been in memory.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 39

CSC312: Operating System II

Steps in handling a page fault

Page replacement

While a user process is executing, a page fault occurs. The operating system determines where the desired page is residing on the disk but then finds that there are no free frames on the free-frame list; all memory is in use.

The operating system has several options at this point.

- o It could terminate the user process
- o The operating system could instead swap out a process, freeing all its frames and reducing the level of multiprogramming.
- o Find one that is not currently being used and free it

□ This is the idea of Page replacement

Page replacement takes the following approach.

- o If no frame is free, we find one that is not currently being used and free it.
- o We can free a frame by writing its contents to swap space and changing the page table (and all other tables) to indicate that the page is no longer in memory

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 40

CSC312: Operating System II

- o We can now use the freed frame to hold the page for which the process faulted.

The above steps are modified as follows:

1. Find the location of the desired page on the disk.
2. Find a free frame:
 - a. If there is a free frame, use it.
 - b. If there is no free frame, use a page-replacement algorithm to select a victim frame.
 - c. Write the victim frame to the disk; change the page and frame tables accordingly.
3. Read the desired page into the newly freed frame; change the page and

frame tables.

4. Continue the user process from where the page fault occurred.

Notice that, if no frames are free, two page transfers (one out and one in) are required. This situation effectively doubles the page-fault service time and increases the effective access time accordingly.

Page replacement

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 41

CSC312: Operating System II

Page replacement Algorithms

There are many different page-replacement algorithms. Every operating system probably has its own replacement scheme.

- o How do we select a particular replacement algorithm?

- In general, we want the one with the lowest page-fault rate.

We evaluate an algorithm by running it on a particular string of memory references and computing the number of page faults.

- o The string of memory references is called a reference string.

To determine the number of page faults for a particular reference string and page-replacement algorithm,

- o we also need to know the number of page frames available.

- o as the number of frames available increases, the number of page faults decreases.

- o adding physical memory increases the number of frames.

We next illustrate several page-replacement algorithms. In doing so, we use the reference string

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

for a memory with three frames.

FIFO Page Replacement

The simplest page-replacement algorithm is a first-in, first-out (FIFO) algorithm.

A FIFO replacement algorithm associates with each page the time when that page was brought into memory.

- o When a page must be replaced, the oldest page is chosen.

Notice that it is not strictly necessary to record the time when a page is brought in.

- o We can create a FIFO queue to hold all pages in memory.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 42

CSC312: Operating System II

- o We replace the page at the head of the queue. When a page is brought into memory, we insert it at the tail of the queue.

Illustration:

For our example reference string, our three frames are initially empty.

- o The first three references (7, 0, 1) cause page faults and are brought into these empty frames.
- o The next reference (2) replaces page 7, because page 7 was brought in first.
- o Since 0 is the next reference and 0 is already in memory, we have no fault for this reference.
- o The first reference to 3 results in replacement of page 0, since it is now first in line. Because of this replacement, the next reference, to 0, will fault.
- o Page 1 is then replaced by page 0.

This process continues as shown in the diagram below. Every time a fault occurs, we show which pages are in our three frames. There are fifteen faults altogether

FIFO page-replacement algorithm

Least Recently Used (LRU) Algorithm.

LRU replacement associates with each page the time of that page's last use. When a page must be replaced,

- o LRU chooses the page that has not been used for the longest period of time.

The result of applying LRU replacement to our example reference string is shown in the diagram below. The LRU algorithm produces twelve faults.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 43

CSC312: Operating System II

LRU page-replacement algorithm

Other replacement algorithms include

- i. Second Chance
- ii. Enhanced Second Chance
- iii. Least Frequently Used (LFU)
- iv. Most Frequently Used (MFU)
- v. Stack algorithm

These algorithms are left as research topics for students.

Hit Ratios: Determining Which Replacement Policy to Use

Using each of the replacement policies to swap pages between main memory and the page file will result in different hit ratios.

- o A hit ratio is the number of times that a page is actually found in main memory, as opposed to the number of page faults generated (requiring the memory manager to retrieve the page from virtual memory).
- o To calculate the hit ratio, we divide the number of non page faults by the total number of page requests (the number of times that data has been sent between the CPU and main memory).
- o Page faults occur when a page is requested by the CPU that is not currently in the page table (in other words, not currently in RAM).

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 44

CSC312: Operating System II

Calculating Hit Ratios

To understand how to calculate hit ratios, we will examine an example that uses a RAM space of three (3) frames, and the following processing sequence:

12131415232415

In these examples, Y = a hit (the page is found in RAM), and N = a page fault (the page must be retrieved from virtual memory)

FIFO Algorithm

LRU Algorithm

Thrashing

Thrashing is a condition where a process is busy swapping pages in and out due to insufficient frames in physical memory.

- o A process is thrashing if it is spending more time paging than executing user instructions.

- o A process is swapped out just before it is needed

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 45

CSC312: Operating System II

Thrashing results in severe performance problems:

- o Low CPU utilization

- o High disk utilization

- o Utilization of other I/O devices

Causes of Thrashing

Thrashing is typically caused by too many processes running at the same time. All the processes will compete for the limited amount of physical memory installed on the computer.

- o If the computer is thrashing, applications may stop responding (or run very slowly).

How to Prevent Thrashing

- The current best practice in implementing a computer facility is to include enough physical memory, whenever possible, to avoid thrashing and swapping.

- o From smart phones through mainframes, providing enough memory to keep all working sets in memory concurrently, except under extreme conditions, gives the best user experience.

- We can limit the effects of thrashing by using a local replacement algorithm (or priority replacement algorithm).

- o With local replacement, if one process starts thrashing, it cannot steal frames from another process and cause the latter to thrash as well.

However, the problem is not entirely solved.

- To prevent thrashing, we must provide a process with as many frames as it

needs.

- o One technique to know the number of frames a process needs is the working-set model.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 46

CSC312: Operating System II

- o The working-set strategy starts by looking at how many frames a process is actually using.

- If a page is in active use, it will be in the working set. If it is no longer being used, it will drop from the working set

Exercises

1. Under what circumstances do page faults occur? Describe the actions taken by the operating system when a page fault occurs.

2. Consider the following page reference string:

1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6

How many page faults would occur for the following replacement algorithms, assuming one, two, three, four, five, six, and seven frames?

Remember that all frames are initially empty, so your first unique pages will cost one fault each.

- i. LRU replacement

- ii. FIFO replacement

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 47

CSC312: Operating System II

2. CPU SCHEDULING

A process is executed until it must wait, typically for the completion of some I/O request.

- o In a simple computer system, the CPU then just sits idle

- o All this waiting time is wasted; no useful work is accomplished.

CPU scheduling is the basis of multi-programmed operating systems. By switching the CPU among processes, the operating system can make the computer more productive.

- o The objective of multiprogramming is to have some process running at all times, to maximize CPU utilization.

- o Several processes are kept in memory at one time.

- o Every time one process has to wait (perhaps for I/O request), another process can take over use of the CPU.

The CPU is, of course, one of the primary computer resources. Thus, its scheduling is central to operating-system design.

- CPU scheduling deals with the problem of deciding which of the processes in the ready queue is to be allocated the CPU.

2.1 Basic Concepts

CPU-I/O Burst Cycle

The success of CPU scheduling depends on CPU-I/O burst cycle since process execution consists of a cycle of CPU execution and I/O wait.

- Processes alternate between these two states.

Process execution begins with a CPU burst. That is followed by an I/O burst, which is followed by another CPU burst, then another I/O burst, and so on (see diagram below)

- CPU burst – process bound to CPU

- I/O burst – process bound to I/O

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 48

CSC312: Operating System II

- CPU Burst time - amount of time the process uses the processor before it is no longer ready

Alternating sequence of CPU and I/O bursts

CPU Scheduler

Whenever the CPU becomes idle, the operating system must select one of the processes in the ready queue to be executed.

- The selection process is carried out by the short-term scheduler, or CPU scheduler.

- The scheduler selects a process from the processes in memory that are ready to execute and allocates the CPU to that process.

- Conceptually, all the processes in the ready queue are lined up waiting for a chance to run on the CPU.

Preemptive & Non-preemptive Scheduling

CPU scheduling decisions may take place when a process:

- Switches from running to waiting state (I/O request).
- Switches from running to ready state (when interrupt occurs).

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 49

CSC312: Operating System II

- Switches from waiting to ready (completion of I/O).

- Terminates

Scheduling under 1 and 4 is non-preemptive:

- Once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU either by terminating or by switching to the waiting state.

Scheduling under 2 and 3 is preemptive:

- Allows a running process to be interrupted by a high priority process

- The process remains in the ready queue until it gets the next opportunity to execute

Preemptive scheduling incurs a cost.

- o Assume that two processes sharing data. One may be in the middle of updating the data when it is preempted and the second process is running. The second process may try to read the data which are currently in an inconsistent state

Dispatcher

Another component involved in the CPU-scheduling function is the dispatcher.

□ Dispatcher module gives control of the CPU to the process selected by the short-term scheduler; this function involves:

- o switching context
- o switching to user mode
- o jumping to the proper location in the user program to restart that program

□ Dispatch latency- time it takes for the dispatcher to stop one process and start another running.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 50

CSC312: Operating System II

2.2 Scheduling Criteria & Optimization

Different CPU-scheduling algorithms have different properties and characteristics and may favor one class of processes over another. The characteristics used for comparison can make a substantial difference in the determination of the best algorithm.

The criteria include:-

- o CPU utilization – keep the CPU as busy as possible (40 % for a lightly loaded system to 90 % for heavily used system)
- o Maximize CPU utilization
- o Throughput – # of processes that complete their execution per time unit (1 process / hour for long processes, 10 process / second for short transactions)
- o Maximize throughput
- o Turnaround time – amount of time it takes to execute a particular process (from time of submission to time of completion)
- o Minimize turnaround time
- o Waiting time – amount of time a process has been waiting in the ready queue
- o Minimize waiting time
- o Response time – amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment)
- o Minimize response time

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 51

CSC312: Operating System II

2.3 Scheduling Algorithms

There are many different CPU-scheduling algorithms. These include:

- g. First-Come, First-Served Scheduling
- h. Shortest-Job-First Scheduling
- i. Priority Scheduling
- j. Round-Robin Scheduling
- k. Multi-level Queue Scheduling
- l. Multilevel feedback Queue Scheduling

First-Come, First-Served Scheduling

The simplest CPU-scheduling algorithm is the first-come, first-served (FCFS) scheduling algorithm.

- o With this scheme, the process that requests the CPU first is allocated the CPU first.
- o The implementation of the FCFS policy is easily managed with a FIFO queue.
- o The average waiting time under the FCFS policy is often quite long.

Consider the following set of processes that arrive at time 0, with the length of the CPU burst given in milliseconds:

Process Burst Time

P1 24

P2 3

P3 3

If the processes arrive in the order P1, P2, P3, and are served in FCFS order, we get the result shown in the following Gantt chart

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 52

CSC312: Operating System II

The waiting time is 0 milliseconds for process P1, 24 milliseconds for process P2, and 27 milliseconds for process P3.

Thus, the average waiting time is $(0 + 24 + 27)/3 = 17$ milliseconds.

If the processes arrive in the order P2, P3, P1, however, the results will be as shown in the following Gantt chart:

Waiting time for P1 = 6, P2 = 0, P3 = 3

Average waiting time is now $(6 + 0 + 3)/3 = 3$ milliseconds.

This reduction is substantial.

Shortest-Job-First Scheduling

Associate with each process the length of its next CPU burst. Use these lengths to schedule the process with the shortest time.

Two schemes:

- o non-preemptive – once CPU given to the process it cannot be pre-empted until completes its CPU burst.
- o preemptive – if a new process arrives with CPU burst length less than remaining time of current executing process, preempt. This scheme is known

as the Shortest-Remaining-Time-First (SRTF).

SJF is optimal – gives minimum average waiting time for a given set of processes.

As an example of SJF scheduling, consider the following set of processes, with the length of the CPU burst given in milliseconds:

Process Burst Time

P1 6

P2 8

P3 7

P4 3

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 53

CSC312: Operating System II

Using non-preemptive SJF scheduling, we would schedule these processes according to the following Gantt chart:

Waiting time: P1 = 3, P2 = 16, P3 = 9, P4 = 0

Average waiting time = $(3 + 16 + 9 + 0)/4 = 7\text{ms}$

By comparison, if we were using the FCFS scheduling scheme, the average waiting time would be 10.25 ms

Consider another example using both preemptive and non-preemptive SJF

Process Arrival Time Burst Time

P1 0 8

P2 1 4

P3 2 9

P4 3 5

Preemptive SJF:

Average waiting time : P1 : $10 - 1 = 9$

P2 : $1 - 1 = 0$

P3 : $17 - 2 = 15$

P4 : $5 - 3 = 2$

AWT = $(9+0+15+2) / 4 = 6.5\text{ ms}$

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 54

CSC312: Operating System II

Non-Preemptive SJF:

AWT = $0 + (8 - 1) + (12 - 3) + (17 - 2) / 4 = 7.75\text{ ms}$

Priority Scheduling

The SJF algorithm is a special case of the general priority-scheduling algorithm. A priority is associated with each process, and the CPU is allocated to the process with the highest priority.

o Equal-priority processes are scheduled in FCFS order.

o An SJF algorithm is simply a priority algorithm where the priority (p) is the

inverse of the (predicted) next CPU burst. The larger the CPU burst, the lower the priority, and vice versa.

As an example, consider the following set of processes, assumed to have arrived at time 0 in the order P1, P2, ..., P5, with the length of the CPU burst given in milliseconds:

Process Burst Time Priority

P1 10 3

P2 1 1

P3 2 4

P4 1 5

P5 5 2

Using priority scheduling, we would schedule these processes according to the following Gantt chart:

$$\text{AWT} = (0 + 1 + 6 + 16 + 18)/5 = 41/5 \\ = 8.2\text{ms}$$

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 55

CSC312: Operating System II

Priority scheduling can be either preemptive or non-preemptive. When a process arrives at the ready queue, its priority is compared with the priority of the currently running process.

□ A preemptive priority scheduling algorithm will preempt the CPU if the priority of the newly arrived process is higher than the priority of the currently running process.

□ A non-preemptive priority scheduling algorithm will simply put the new process at the head of the ready queue.

A major problem with priority scheduling algorithms is indefinite blocking, or starvation.

o A process that is ready to run but waiting for the CPU can be considered blocked.

o A priority scheduling algorithm can leave some low priority processes waiting indefinitely

Round-Robin Scheduling

The round-robin (RR) scheduling algorithm is designed especially for timesharing systems. It is similar to FCFS scheduling, but preemption is added to switch between processes. A small unit of time, called a time quantum (or time slice), is defined.

o The ready queue is treated as a circular queue.

o The CPU scheduler goes around the ready queue, allocating the CPU to each process for a time interval of up to 1 time quantum.

The average waiting time under the RR policy is often long. Consider the following set of processes that arrive at time 0, with the length of the CPU burst given in

milliseconds:

Process Burst Time

P1 24

P2 3

P3 3

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 56

CSC312: Operating System II

Using Time quantum = 4 ms

The Gantt chart is shown as follows:

Waiting time for P1 = 26 - 20 = 6

P2 = 4

P3 = 7

AWT = $(6+4+7) / 3 = 5.66$ ms

The performance of the RR algorithm depends heavily on the size of the time-quantum.

- o If time-quantum is very large (infinite) then RR policy is same as FCFS policy.

- o If time quantum is very small, RR approach is called processor sharing and appears to the users as though each of n process has its own processor running at 1/n the speed of real processor.

Multilevel Queue Scheduling

A multilevel queue-scheduling algorithm partitions the ready queue into several separate queues.

- o Foreground (interactive)

- o Background (batch)

- o The foreground (interactive) queue may have absolute priority over the background (batch) queue

These two types of processes have different response-time requirements and so may have different scheduling needs. Each queue has its own scheduling algorithm

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 57

CSC312: Operating System II

- o foreground queue - RR

- o background queue - FCFS

The processes are permanently assigned to one queue, generally based on some property of the process, such as

- o memory size,

- o process priority, or

- o process type.

Scheduling must be done between the queues

- o Fixed priority scheduling; (i.e., serve all from foreground then from background).

- Danger of starvation.

- o Time slice - each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR, 20% to background in FCFS

Example of a multilevel queue scheduling algorithm with five queues

1. System processes
2. Interactive processes
3. Interactive editing processes
4. Batch processes
5. Student processes

Multilevel queue scheduling

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 58

CSC312: Operating System II

Each queue has absolute priority over lower-priority queue.

No process in the batch queue, for example, could run unless the queues for system processes, interactive processes, and interactive editing processes were all empty.

- o If an interactive editing process entered the ready queue while a batch process was running, the batch process would be preempted.

Multilevel Feedback Queue Scheduling

Multilevel feedback queue scheduling allows a process to move between queues.

The idea is to separate processes with different CPU-burst characteristics.

- o If a process uses too much CPU time, it will be moved to a lower-priority queue.

- o This scheme leaves I/O-bound and interactive processes in the higher-priority queues.

- o Similarly, a process that waits too long in a lower priority queue may be moved to a higher-priority queue.

- o This form of aging prevents starvation.

For example, consider a multilevel feedback queue scheduler with three queues, numbered from 0 to 2

Three queues:

Q0 - RR with time quantum 8 milliseconds

Q1 - RR time quantum 16 milliseconds

Q2 - FCFS

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 59

CSC312: Operating System II

Multilevel feedback queues.

Scheduling:

- o A new job enters queue Q0. When it gains CPU, job receives 8 milliseconds.

If it exhausts 8 milliseconds and does not complete, job is moved to queue

Q1.

- o At Q1 the job receives 16 additional milliseconds. If it still does not complete, it is preempted and moved to queue Q2.

The scheduler first executes all processes in queue 0. Only when queue 0 is empty will it execute processes in queue 1.

- o Similarly, processes in queue 2 will be executed only if queues 0 and 1 are empty.

- o A process that arrives for queue 1 will preempt a process in queue 2.

- o A process that arrives for queue 0 will, in turn, preempt a process in queue 1.

A multilevel feedback queue scheduler is defined by the following parameters:

- o The number of queues

- o The scheduling algorithm for each queue

- o The method used to determine when to upgrade a process to a higher priority queue

- o The method used to determine when to demote a process to a lower-priority queue

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 60

CSC312: Operating System II

- o The method used to determine which queue a process will enter when that process needs service

Exercises

1. Explain the difference between preemptive and non-preemptive scheduling.

2. Suppose that the following processes arrive for execution at the times indicated. Each process will run for the amount of time listed. In answering the questions, use non-preemptive scheduling, and base all decisions on the information you have at the time the decision must be made.

Process Arrival Time Burst Time

P1 0.0 8

P2 0.4 4

P3 1.0 1

- a. What is the average turnaround time for these processes with the FCFS scheduling algorithm?

- b. What is the average turnaround time for these processes with the SJF scheduling algorithm?

3. Why is it important for the scheduler to distinguish I/O-bound programs from CPU-bound programs?

4. Consider the following set of processes, with the length of the CPU burst

given in milliseconds:

Process Burst Time Priority

P1 2 2

P2 1 1

P3 8 4

P4 4 2

P5 5 3

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 61

CSC312: Operating System II

The processes are assumed to have arrived in the order P1, P2, P3, P4, P5, all at time 0.

- a. Draw four Gantt charts that illustrate the execution of these processes using the following scheduling algorithms: FCFS, SJF, nonpreemptive priority (a larger priority number implies a higher priority), and RR (quantum = 2).
- b. What is the turnaround time of each process for each of the scheduling algorithms in part a?
- c. What is the waiting time of each process for each of these scheduling algorithms?
- d. Which of the algorithms results in the minimum average waiting time (over all processes)?

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 62

CSC312: Operating System II

3. Mutual Exclusion & Synchronization

3.1 Race Condition, Critical Section, and Mutual Exclusion

Processes frequently need to communicate with other processes. A cooperating process is one that can affect or be affected by other processes executing in the system. Cooperating processes can either directly share a logical address space (that is, both code and data) or be allowed to share data only through files or messages. However, sometimes a process has to access shared memory or files, or do other critical things that can lead to race condition

o A race condition is a situation in which multiple threads or processes read and write a shared data item and the final result depends on the relative timing of their execution

A race condition occurs when several processes access and manipulate the same data concurrently and the outcome of the execution depends on the particular order in which the access takes place. That part of the program where the shared memory is accessed is called the critical region or critical section.

o A critical section is a section of code within a process that requires access to shared resources and that must not be executed while another process is in a corresponding section of code

□ Each process must request permission to enter its

critical section. The section of code implementing this request is the entry section. The critical section may be followed by an exit section. The remaining code is the remainder section.

The general structure is illustrated below

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 63

CSC312: Operating System II

The key to preventing trouble here and in many other situations involving shared memory, shared files, and shared everything else is to find some way to prohibit more than one process from reading and writing the shared data at the same time. Put in other words, what is needed is mutual exclusion, that is, some way of making sure that if one process is using a shared variable or file, the other processes will be excluded from doing the same thing.

o Mutual exclusion is a requirement that when one process is in a critical section that accesses shared resources, no other process may be in a critical section that accesses any of those shared resources

The critical-section problem is to design a protocol that the processes can use to cooperate. Each process must request permission to enter its critical section.

We need four conditions to hold to have a good solution:

No two processes may be simultaneously inside their critical regions.

No assumptions may be made about speeds or the number of CPUs.

No process running outside its critical region may block any process.

No process should have to wait forever to enter its critical region.

Figure 3.1 below illustrates how mutual exclusion can be achieved using critical section. Here process A enters its critical region at time T1. A little later, at time T2 process B attempts to enter its critical region but fails because another process is already in its critical region and can allow only one at a time. Consequently, B is temporarily suspended until time T3 when A leaves its critical region, allowing B to enter immediately. Eventually B leaves (at T4) and it is back to the original situation with no processes in their critical regions.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 64

CSC312: Operating System II

Fig3.1 Mutual exclusion using critical regions

3.2 Solutions to Critical Section Problem

A solution to the critical-section problem must satisfy the following three requirements:

□ Mutual exclusion. If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.

□ Progress. If no process is executing in its critical section and some

processes wish to enter their critical sections, then only those processes that are not executing in their remainder sections can participate in deciding which will enter its critical section next, and this selection cannot be postponed indefinitely.

□ Bounded waiting. There exists a bound, or limit, on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 65

CSC312: Operating System II

Two general approaches are used to handle critical sections in operating systems: preemptive kernels and non-preemptive kernels.

□ A preemptive kernel allows a process to be preempted while it is running in kernel mode.

o Preemptive kernels must be carefully designed to ensure that shared kernel data are free from race conditions.

□ A non-preemptive kernel does not allow a process running in kernel mode to be preempted; a kernel-mode process will run until it exits kernel mode, blocks, or voluntarily yields control of the CPU.

o Obviously, a non-preemptive kernel is essentially free from race conditions on kernel data structures, as only one process is active in the kernel at a time

3.2.1 Peterson's Solution

Peterson solution is a classic software-based solution to the critical-section problem. It provides a good algorithmic description of solving the critical-section problem.

□ Peterson's solution is restricted to two processes that alternate execution between their critical sections and remainder sections

Fig. 3.2.The structure of process P_i in Peterson solution

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 66

CSC312: Operating System II

The variable `turn` indicates whose turn it is to enter its critical section. That is, if `turn == i`, then process P_i is allowed to execute in its critical section. The `flag` array is used to indicate if a process is ready to enter its critical section. For example, if `flag[i]` is true, this value indicates that P_i is ready to enter its critical section. To enter the critical section, process P_i first sets `flag[i]` to be true and then sets `turn` to the value `j`, thereby asserting that if the other process wishes to enter the critical section, it can do so.

o If both processes try to enter at the same time, `turn` will be set to both `i` and `j` at roughly the same time.

- o Only one of these assignments will last; the other will occur but will be overwritten immediately.
- o The eventual value of turn determines which of the two processes is allowed to enter its critical section first.

Because of the way modern computer architectures perform basic machine-language instructions, such as load and store, there are no guarantees that Peterson's solution will work correctly on such architectures

3.2.2 Mutex Locks

Operating-systems designers build software tools to solve the critical-section problem. The simplest of these tools is the mutex lock. (In fact, the term mutex is short for mutual exclusion.)

A mutex is a shared variable that can be in one of two states: unlocked or locked. Consequently, only 1 bit is required to represent it, but in practice an integer often is used, with 0 meaning unlocked and all other values meaning locked

The mutex lock is used to protect critical regions and thus prevent race conditions.

- o That is, a process must acquire the lock before entering a critical section; it releases the lock when it exits the critical section.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 67

CSC312: Operating System II

- o The acquire() function acquires the lock, and the release() function releases the lock.

A mutex lock has a boolean variable available whose value indicates if the lock is available or not. If the lock is available, a call to acquire() succeeds, and the lock is then considered unavailable. A process that attempts to acquire an unavailable lock is blocked until the lock is released.

The definition of acquire() is as follows:

Figure 3.3 illustrates the Solution to the critical-section problem using mutex locks.

Fig 3.3 Solution to the critical-section problem using Mutex locks

The definition of release() is as follows:

```
release() {
    available = true;
```

The main disadvantage of the implementation given here is that it requires busy waiting.

- o While a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the call to acquire().

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 68

CSC312: Operating System II

- o In fact, this type of mutex lock is also called a spinlock because the process “spins” while waiting for the lock to become available.
- o This continual looping is clearly a problem in a real multiprogramming system, where a single CPU is shared among many processes.
- o Busy waiting wastes CPU cycles that some other process might be able to use productively

3.2.3 Semaphores

A semaphore is another synchronization tool used by processes to synchronize their activities. A semaphore is an integer variable used for signaling among processes. It is accessed only through two standard atomic operations: wait() and signal() or sleep and wakeup as used in some text.

The definition of wait() is as follows:

```
// execute to use a resource
wait(S) {
while (S <= 0)
// busy wait
S--;
```

The definition of signal() is as follows:

```
// execute to release a resource
signal(S) {
S++;
```

All modifications to the integer value of the semaphore in the wait() and signal() operations must be executed indivisibly.

- o That is, when one process modifies the semaphore value, no other process can simultaneously modify that same semaphore value.
- o . In addition, in the case of wait(S), the testing of the integer value of S ($S \leq 0$), as well as its possible modification (S--), must be executed without interruption

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 69

CSC312: Operating System II

□ Binary & Counting Semaphore

Operating systems often distinguish between counting and binary semaphores.

The value of a binary semaphore can range only between 0 and 1. Thus, binary semaphores behave similarly to mutex locks. In fact, on systems that do not provide mutex locks, binary semaphores can be used instead for providing mutual exclusion.

The value of a counting semaphore can range over an unrestricted domain.

Counting semaphores can be used to control access to a given resource consisting of a finite number of instances. The semaphore is initialized to the number of resources available. Each process that wishes to use a resource performs a wait() operation on

the semaphore (thereby decrementing the count).

When a process releases a resource, it performs a `signal()` operation (incrementing the count). When the count for the semaphore goes to 0, all resources are being used. After that, processes that wish to use a resource will block until the count becomes greater than 0.

3.2.3.1 Semaphore Implementation with Blocking

The definitions of the `wait()` and `signal()` semaphore operations just described suffers the same problem of busy waiting just as in mutex locks.

To overcome the need for busy waiting, the definition of the `wait()` and `signal()` operations can be modified as follows:

- o When a process executes the `wait()` operation and finds that the semaphore value is not positive, it must wait.
- o However, rather than engaging in busy waiting, the process can block itself.
- o The block operation places a process into a waiting queue associated with the semaphore, and the state of the process is switched to the waiting state.
- o Then control is transferred to the CPU scheduler, which selects another process to execute.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 70

CSC312: Operating System II

- o A process that is blocked, waiting on a semaphore `S`, should be restarted when some other process executes a `signal()` operation.
- The process is restarted by a `wakeup()` operation, which changes the process from the waiting state to the ready state. The process is then placed in the ready queue

Now, the `wait()` semaphore operation can be defined as

and the `signal()` semaphore operation can be defined as

The `block()` operation suspends the process that invokes it. The `wakeup(P)` operation resumes the execution of a blocked process `P`. These two operations are provided by the operating system as basic system calls.

3.2.3.2 Problems of Implementing Semaphore with blocking

1. Deadlocks

The implementation of a semaphore with a waiting queue may result in a situation where two or more processes are waiting indefinitely for an event that can be caused only by one of the waiting processes. The event in question is the

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 71

CSC312: Operating System II

execution of a `signal()` operation. When such a state is reached, these processes are

said to be deadlocked.

- o A set of processes is in a deadlocked state when every process in the set is waiting for an event that can be caused only by another process in the set.
- o The events with which are mainly of concerned here are resource acquisition and release

Illustration of Deadlock

To illustrate this, consider a system consisting of two processes, P0 and P1, each accessing two semaphores, S and Q, set to the value 1:

Suppose that P0 executes wait(S) and then P1 executes wait(Q). When P0 executes wait(Q), it must wait until P1 executes signal(Q).

Similarly, when P1 executes wait(S), it must wait until P0 executes signal(S).

Since these signal() operations cannot be executed, P0 and P1 are deadlocked.

2. Starvation

Another problem related to deadlocks is indefinite blocking or starvation, a situation in which processes wait indefinitely within the semaphore.

- o Indefinite blocking may occur if a process is removed from the list associated with a semaphore in LIFO (last-in, first-out) order.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 72

CSC312: Operating System II

Exercises

1. Explain why spinlocks are not appropriate for single-processor systems yet are often used in multiprocessor systems.
2. Show that, if the wait() and signal() semaphore operations are not executed atomically, then mutual exclusion may be violated.
3. Illustrate how a binary semaphore can be used to implement mutual exclusion among n processes.
4. Race conditions are possible in many computer systems. Consider a banking system that maintains an account balance with two functions: deposit(amount) and withdraw(amount). These two functions are passed the amount that is to be deposited or withdrawn from the bank account balance. Assume that a husband and wife share a bank account. Concurrently, the husband calls the withdraw() function and the wife calls deposit(). Describe how a race condition is possible and what might be done to prevent the race condition from occurring.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 73

CSC312: Operating System II

4. Classical Problems of Synchronization

There are numbers of synchronization problems as examples of a large class of concurrency-control problems that are used for testing nearly every newly proposed synchronization scheme. This section presents such examples.

4.1. The Producer-Consumer Problem

The Producer-Consumer Problem (also known as bounded-buffer problem) is commonly used to illustrate the power of synchronization primitives. Two processes share a common, fixed-size buffer. One of them, the producer, puts information into the buffer, and the other one, the consumer, takes it out.

Trouble arises when the producer wants to put a new item in the buffer, but it is already full. The solution is for the producer to go to sleep, to be awakened when the consumer has removed one or more items. Similarly, if the consumer wants to remove an item from the buffer and sees that the buffer is empty, it goes to sleep until the producer puts something in the buffer and wakes it up.

This approach sounds simple enough, but it leads to race conditions. To keep track of the number of items in the buffer, a variable, count is needed.

- o If the maximum number of items the buffer can hold is N, the producer's code will first test to see if count is N.

- o If it is, the producer will go to sleep; if it is not, the producer will add an item and increment count.

The consumer's code is similar: first test count to see if it is 0.

- o If it is, go to sleep; if it is nonzero, remove an item and decrement the counter.

- o Each of the processes also tests to see if the other should be awakened, and if so, wakes it up.

The code for the producer and consumer process is shown in Figure 4.1

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 74

CSC312: Operating System II

Fig 4.1 The producer-consumer problem with a fatal race condition

4.1.1 Solving the Producer-Consumer Problem Using Semaphores

The producer and consumer processes share the following data structures:

```
int n;
semaphore mutex = 1;
semaphore empty = n;
semaphore full = 0
```

It is assumed that the pool consists of n buffers, each capable of holding one item. The mutex semaphore provides mutual exclusion for accesses to the buffer pool and is initialized to the value 1. The empty and full semaphores count the number of empty and full buffers. The semaphore empty is initialized to the value n; the semaphore full is initialized to the value 0.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 75

CSC312: Operating System II

- o The full and empty semaphores are needed to guarantee that certain event sequences do or do not occur.

o In this case, they ensure that the producer stops running when the buffer is full, and that the consumer stops running when it is empty.

The code for the producer process is shown in Figure 4.2, and the code for the consumer process is shown in Figure 4.3

```
do {
/* produce an item in next produced */
wait(empty);
wait(mutex);
/* add next produced to the buffer */
signal(mutex);
signal(full);
} while (true);
```

Fig 4.2 The structure of the producer process

```
do {
wait(full);
wait(mutex);
/* remove an item from buffer to next consumed */
signal(mutex);
signal(empty);
/* consume the item in next consumed */
} while (true);
```

Fig 4.3 The structure of the consumer process.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 76

CSC312: Operating System II

4.2 The Readers – Writers Problem

Suppose that a database is to be shared among several concurrent processes.

Some of these processes may want only to read the database, whereas others may want to update (that is, to read and write) the database. We distinguish between these two types of processes by referring to the former as readers and to the latter as writers.

Obviously, if two readers access the shared data simultaneously, no adverse effects will result. However, if a writer and some other process (either a reader or a writer) access the database simultaneously, chaos may ensue.

To ensure that these difficulties do not arise, it is required that the writers have exclusive access to the shared database while writing to the database. This synchronization problem is referred to as the readers – writers problem.

The readers – writers problem has several variations, all involving priorities.

The simplest one, referred to as the first readers – writers problem,

□ requires that no reader be kept waiting unless a writer has already obtained permission to use the shared object.

□ In other words, no reader should wait for other readers to finish simply because a writer is waiting.

The second readers – writers problem

□ requires that, once a writer is ready, that writer perform its write as soon as possible.

□ In other words, if a writer is waiting to access the object, no new readers may start reading.

A solution to either problem may result in starvation.

o In the first case, writers may starve;

o In the second case, readers may starve.

o For this reason, other variants of the problem have been proposed.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 77

CSC312: Operating System II

In the solution to the first readers – writers problem, the reader processes share the following data structures:

```
semaphore rw mutex = 1;
```

```
semaphore mutex = 1;
```

```
int read count = 0;
```

The semaphores mutex and rw mutex are initialized to 1; read count is initialized to 0.

□ The semaphore rw mutex is common to both reader and writer processes.

□ The mutex semaphore is used to ensure mutual exclusion when the variable read count is updated.

□ The read count variable keeps track of how many processes are currently reading the object.

□ The semaphore rw mutex functions as a mutual exclusion semaphore for the writers.

□ It is also used by the first or last reader that enters or exits the critical section.

□ It is not used by readers who enter or exit while other readers are in their critical sections.

The code for a writer process is shown in Figure 4.4

```
do {
    wait(rw mutex);
    /* writing is performed */
    signal(rw mutex);
} while (true);
```

Fig 4.4 The structure of a writer process.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 78

CSC312: Operating System II

The code for a reader process is shown in Figure 4.5.

```
do {
wait(mutex);
read count++;
if (read count == 1)
wait(rw mutex);
signal(mutex);
/* reading is performed */
wait(mutex);
read count--;
if (read count == 0)
signal(rw mutex);
signal(mutex);
} while (true);
```

Fig 4.5. The structure of a reader process

Note that:

- o if a writer is in the critical section and n readers are waiting, then one reader is queued on `rw mutex`, and $n - 1$ readers are queued on `mutex`.

- o when a writer executes `signal(rw mutex)`, we may resume the execution of either the waiting readers or a single waiting writer.

- The selection is made by the scheduler.

The readers - writers problem and its solutions have been generalized to provide reader - writer locks on some systems. Acquiring a reader - writer lock requires specifying the mode of the lock: either read or write access.

- When a process wishes only to read shared data, it requests the reader - writer lock in read mode.

- A process wishing to modify the shared data must request the lock in write mode.

- Multiple processes are permitted to concurrently acquire a reader - writer lock in read mode, but only one process may

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 79

CSC312: Operating System II

acquire the lock for writing, as exclusive access is required for writers.

Reader - writer locks are most useful in the following situations:

- In applications where it is easy to identify which processes only read shared data and which processes only write shared data.

- In applications that have more readers than writers.

4.3. The Dining-Philosophers Problem

The dining-philosophers problem is considered a classic synchronization

problem that presents a simple representation of the need to allocate several resources among several processes in a deadlock-free and starvation-free manner. The problem can be stated quite simply as follows. Five philosophers are seated around a circular table. Each philosopher has a plate of spaghetti. The spaghetti is so slippery that a philosopher needs two forks to eat it. Between each pair of plates is one fork.

The life of a philosopher consists of alternating periods of eating and thinking. When a philosopher gets sufficiently hungry, she tries to acquire her left and right forks, one at a time, in either order. If successful in acquiring two forks, she eats for a while, then puts down the forks, and continues to think.

The layout of the table is illustrated in Figure 4.6 using rice for spaghetti and chopsticks for forks.

Fig 4.6 Lunch time in the Philosophy Department

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 80

CSC312: Operating System II

One simple solution is to represent each chopstick with a semaphore. A philosopher tries to grab a chopstick by executing a wait() operation on that semaphore. She releases her chopsticks by executing the signal() operation on the appropriate semaphores.

Thus, the shared data are

semaphore chopstick[5];

where all the elements of chopstick are initialized to 1

The structure of philosopher i is shown in Figure 4.7

```
do {
    wait(chopstick[i]);
    wait(chopstick[(i+1) % 5]);
    /* eat for awhile */
    signal(chopstick[i]);
    signal(chopstick[(i+1) % 5]);
    /* think for awhile */
} while (true);
```

Figure 4.7 The structure of philosopher i.

Although this solution guarantees that no two neighbors are eating simultaneously, it nevertheless must be rejected because it could create a deadlock.

□ Suppose that all five philosophers become hungry at the same time and each grabs her left chopstick. All the elements of chopstick will now be equal to 0.

□ When each philosopher tries to grab her right chopstick, she will be delayed forever.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 81

CSC312: Operating System II

4.3.1 Dining-Philosophers Solution Using Monitors

A monitor is a collection of procedures, variables, and data structures that are all grouped together in a special kind of module or package.

o Processes may call the procedures in a monitor whenever they want to, but they cannot directly access the monitor's internal data structures from procedures declared outside the monitor

The monitor solution imposes the restriction that a philosopher may pick up her chopsticks only if both of them are available. To code this solution, we need to distinguish among three states in which we may find a philosopher.

For this purpose, we introduce the following data structure:

```
enum {THINKING, HUNGRY, EATING} state[5];
```

Philosopher i can set the variable $state[i] = EATING$ only if her two neighbors are not eating: $(state[(i+4) \% 5] \neq EATING)$ and $(state[(i+1) \% 5] \neq EATING)$.

Figure 4.8 shows a monitor solution to the dining-philosopher problem
monitor DiningPhilosophers

```
enum {THINKING, HUNGRY, EATING} state[5];
condition self[5];

void pickup(int i) {
    state[i] = HUNGRY;
    test(i);
    if (state[i] != EATING)
        self[i].wait();

    void putdown(int i) {
        state[i] = THINKING;
        test((i + 4) % 5);
        test((i + 1) % 5);

        void test(int i) {
            if ((state[(i + 4) % 5] != EATING) && (state[i] ==
                HUNGRY) &&
                (state[(i + 1) % 5] != EATING)) {
                state[i] = EATING;
                self[i].signal();
```

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 82

CSC312: Operating System II

```
initialization code() {
    for (int i = 0; i < 5; i++)
        state[i] = THINKING;
```

Fig 4.8 A monitor solution to the dining-philosopher problem

The condition `self[5]` allows philosopher `i` to delay herself when she is hungry but is unable to obtain the chopsticks she needs.

The distribution of the chopsticks is controlled by the monitor

DiningPhilosophers, whose definition is shown in Figure 4.8.

- Each philosopher, before starting to eat, must invoke the operation `pickup()`.

- This act may result in the suspension of the philosopher process.

- After the successful completion of the operation, the philosopher may eat.

- Following this, the philosopher invokes the `putdown()` operation.

- Thus, philosopher `i` must invoke the operations `pickup()` and `putdown()` in the following sequence:

```
DiningPhilosophers.pickup(i);
```

```
eat
```

```
DiningPhilosophers.putdown(i);
```

It is easy to show that this solution ensures that no two neighbors are eating simultaneously and that no deadlocks will occur. However, it is possible for a philosopher to starve to death. The monitor mechanism can be implemented using semaphores.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 83

CSC312: Operating System II

Exercises

The readers and writers problem can be formulated in several ways with regard to which category of processes can be started when. Carefully describe three different variations of the problem, each one favoring (or not favoring) some category of processes. For each variation, specify what happens when a reader or a writer becomes ready to access the database, and what happens when a process is finished

Compiled by Orhionkpaiyo B. C. [December 2019] Updated January 2023 Page 84